

ECE 382N-Sec (FA26):

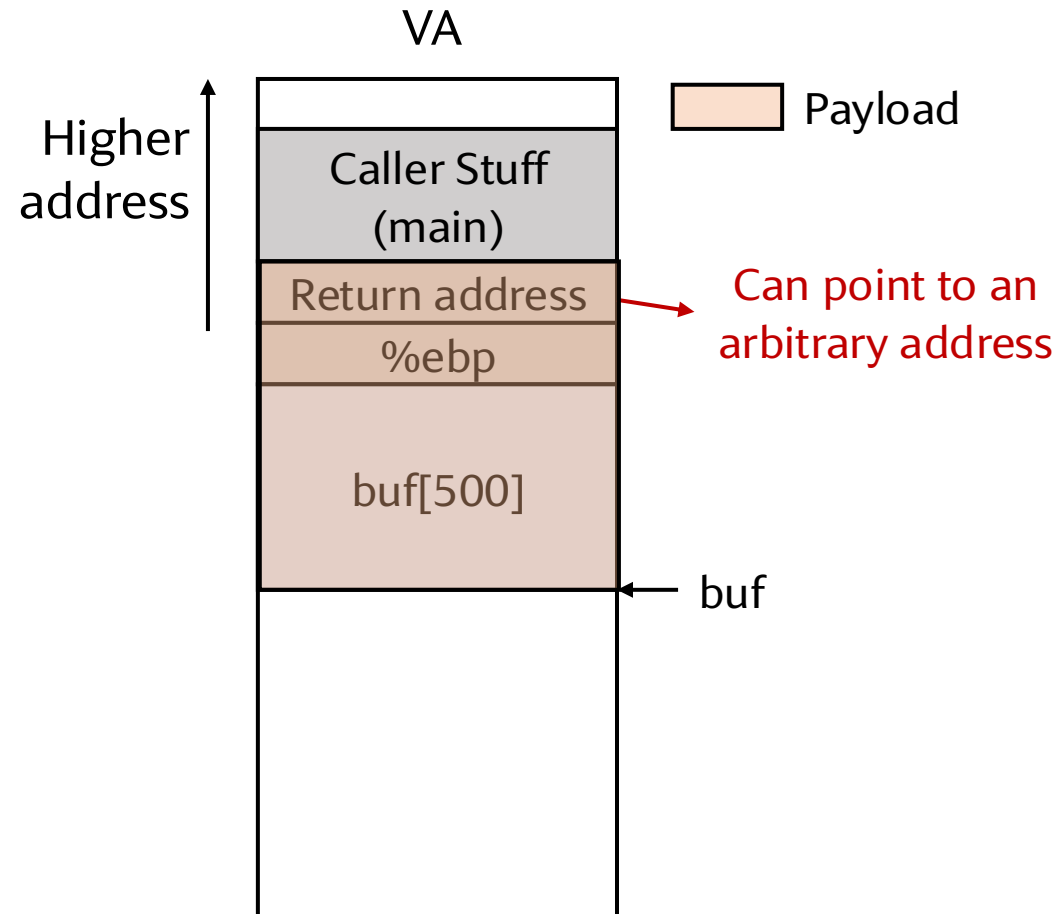
# L10: Memory Safety Attacks

Neil Zhao

[neil.zhao@utexas.edu](mailto:neil.zhao@utexas.edu)

# Buffer Overflow Attacks

Based on <https://www.youtube.com/watch?v=1S0aBV-Waao>



```
int main(int argc, char **argv) {  
    char buf[500];
```

```
    strcpy(buf, argv[1]);  
    return 0;
```

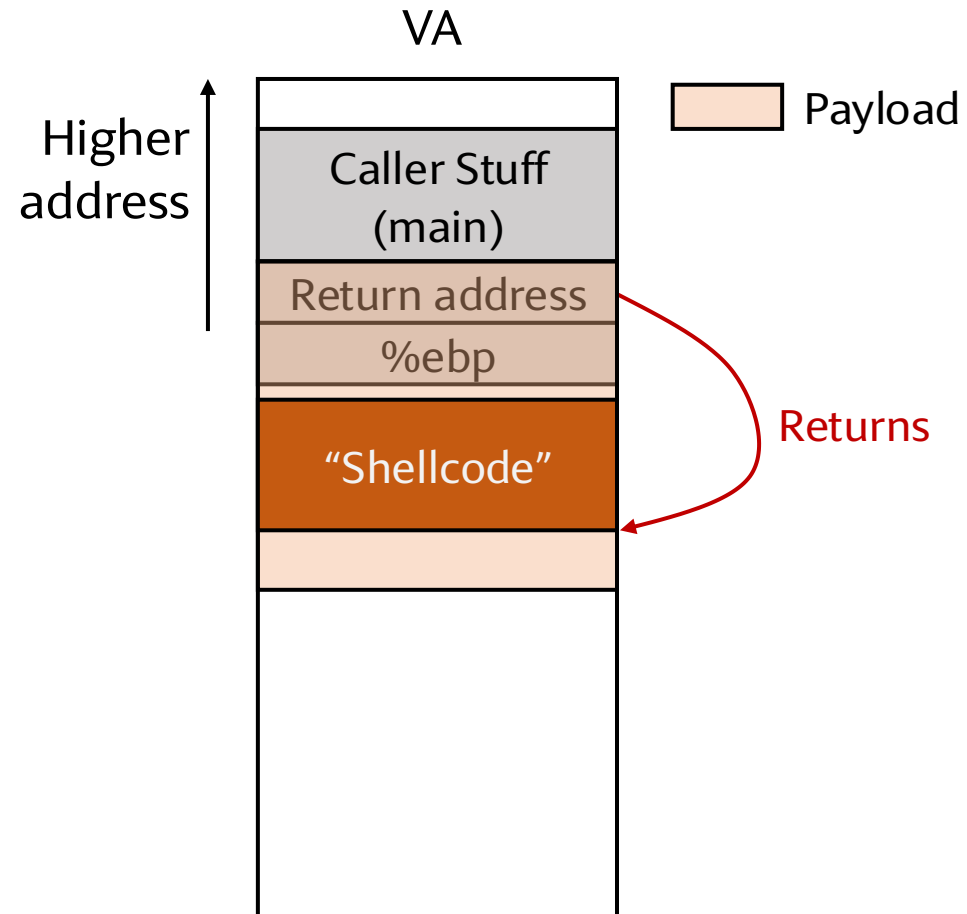
```
}
```

No bounds check

argv[1] is under the attacker's control:  
Set argv[1] to "a long long string ..."

# Buffer Overflow Attacks

Based on <https://www.youtube.com/watch?v=1S0aBV-Waao>



```
int main(int argc, char **argv) {  
    char buf[500];
```

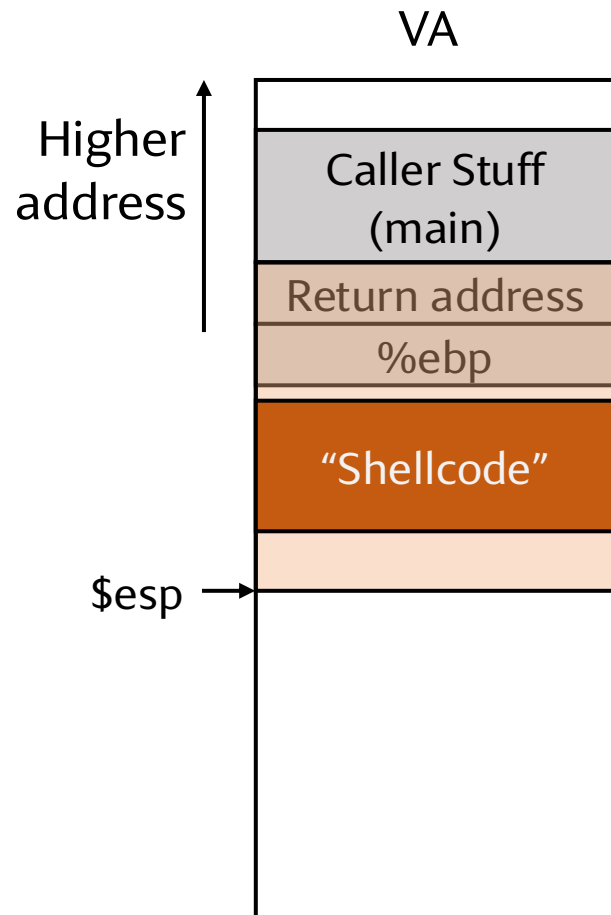
```
    strcpy(buf, argv[1]);  
    return 0;
```

```
}
```

No bounds check

argv[1] is under the attacker's control:  
Set argv[1] to "a long long string ..."

# Understanding The Shell Code



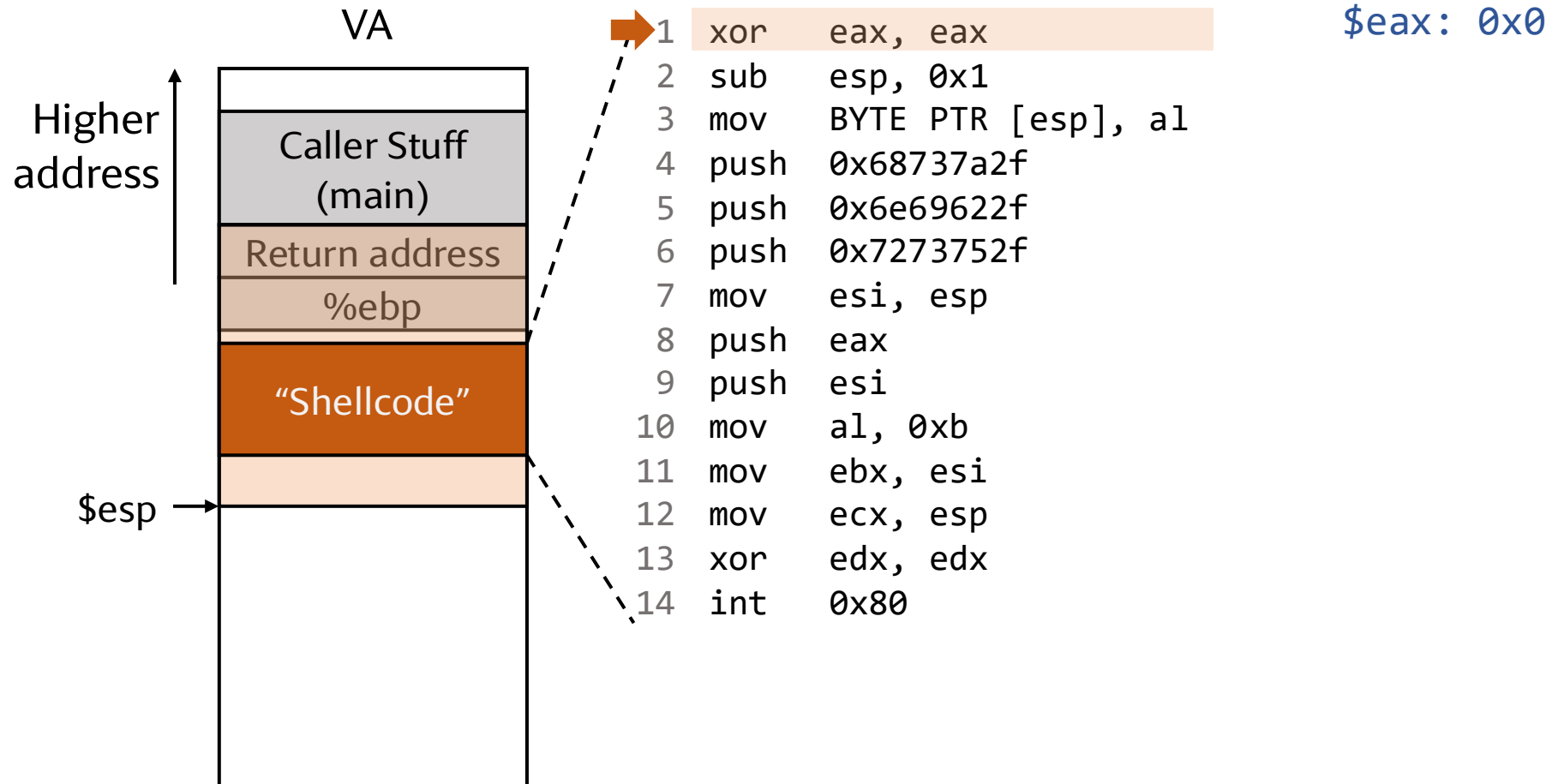
```
1  xor    eax, eax
2  sub    esp, 0x1
3  mov    BYTE PTR [esp], al
4  push   0x68737a2f
5  push   0x6e69622f
6  push   0x7273752f
7  mov    esi, esp
8  push   eax
9  push   esi
10 mov    al, 0xb
11 mov    ebx, esi
12 mov    ecx, esp
13 xor    edx, edx
14 int    0x80
```

Prepare the arguments

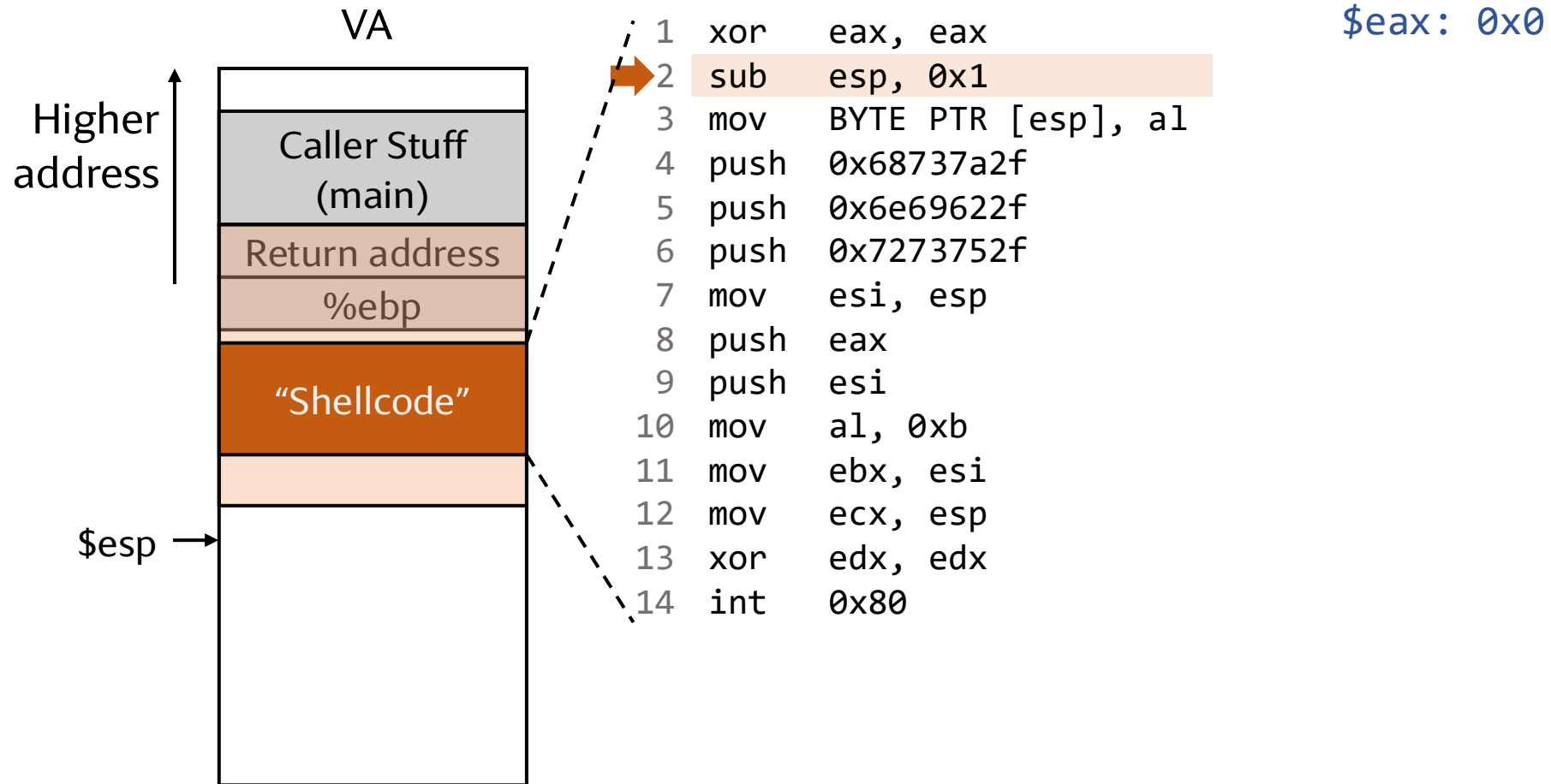
Make the system call to launch a shell

```
execve("/usr/bin/zsh",  
{"/usr/bin/zsh", NULL}, NULL);
```

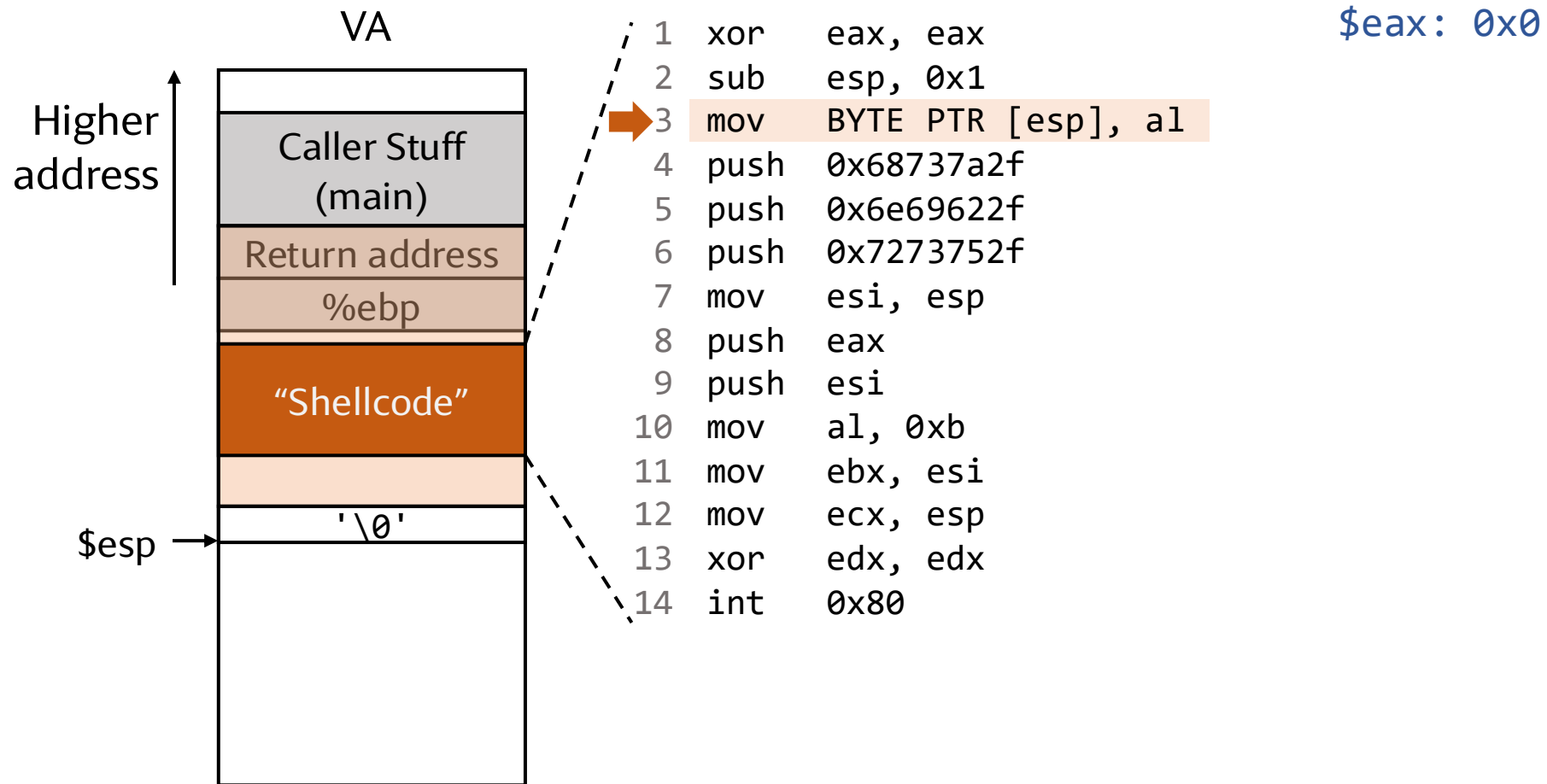
# Understanding The Shell Code



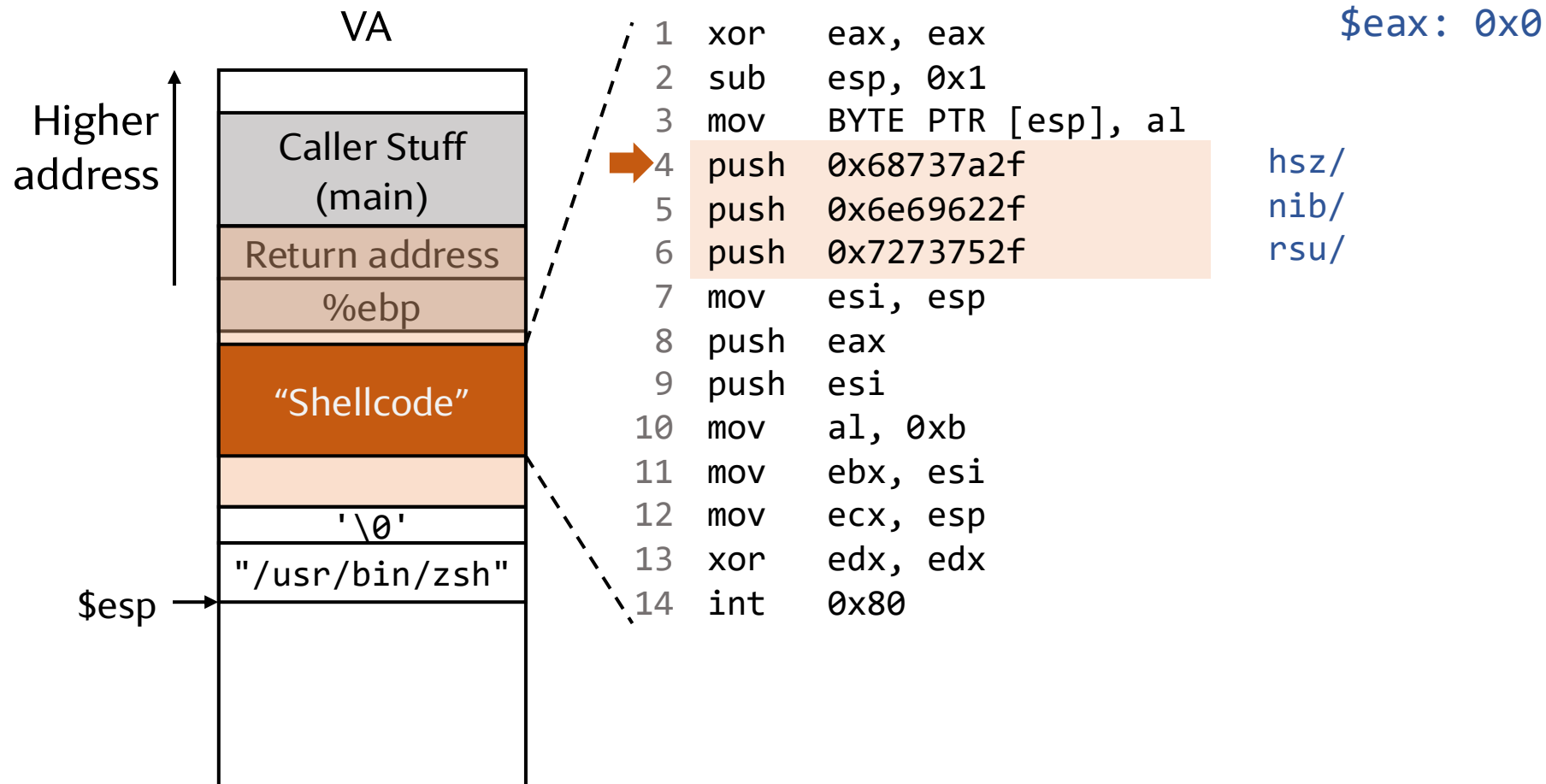
# Understanding The Shell Code



# Understanding The Shell Code

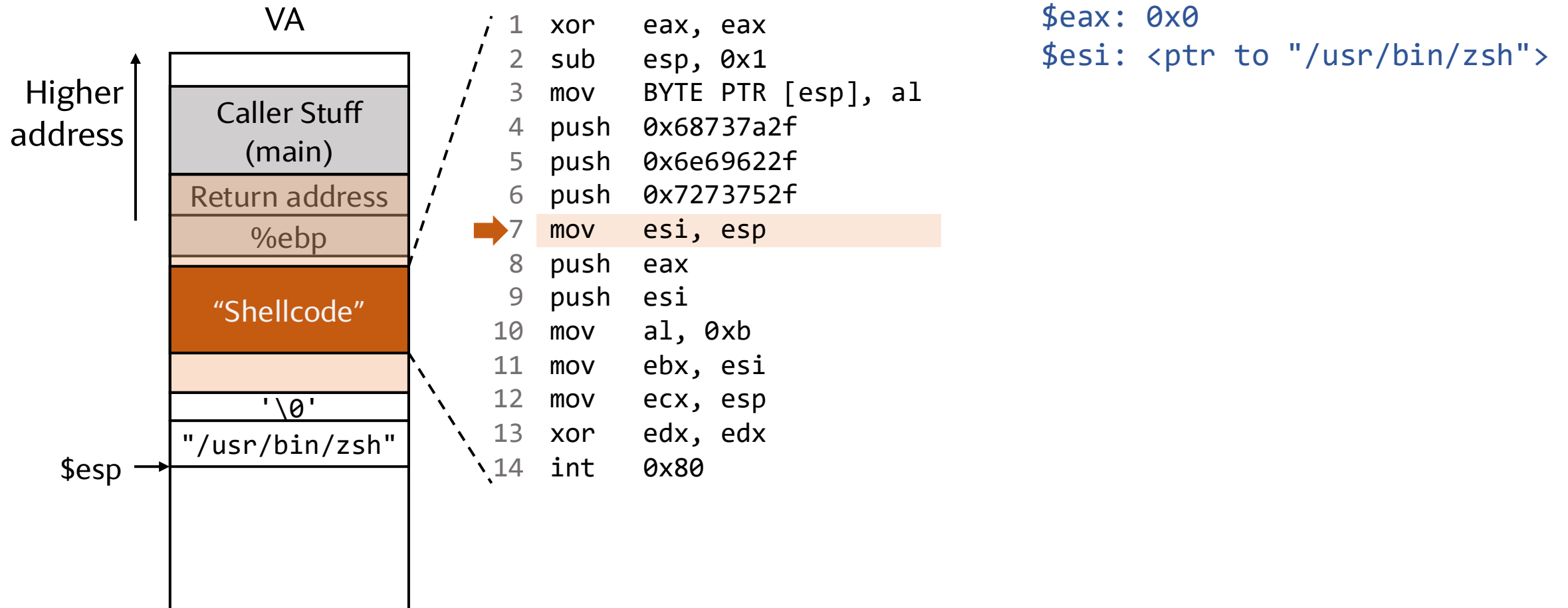


# Understanding The Shell Code

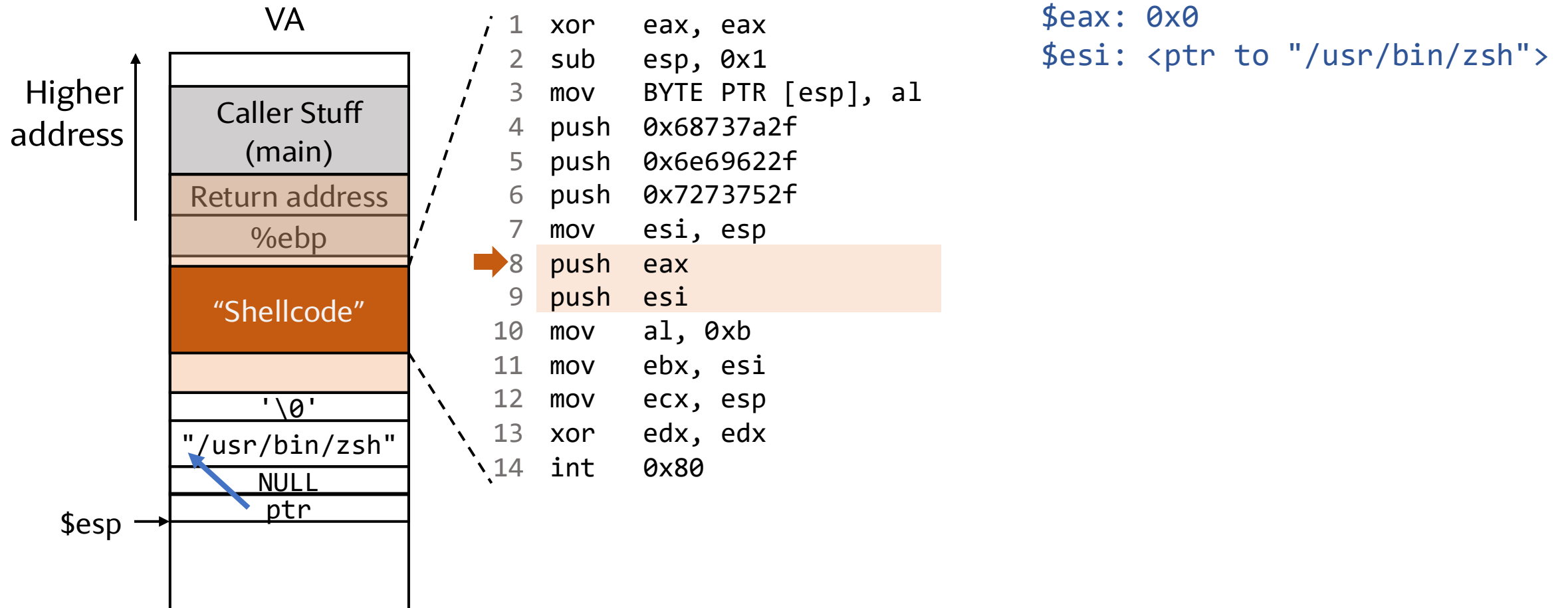




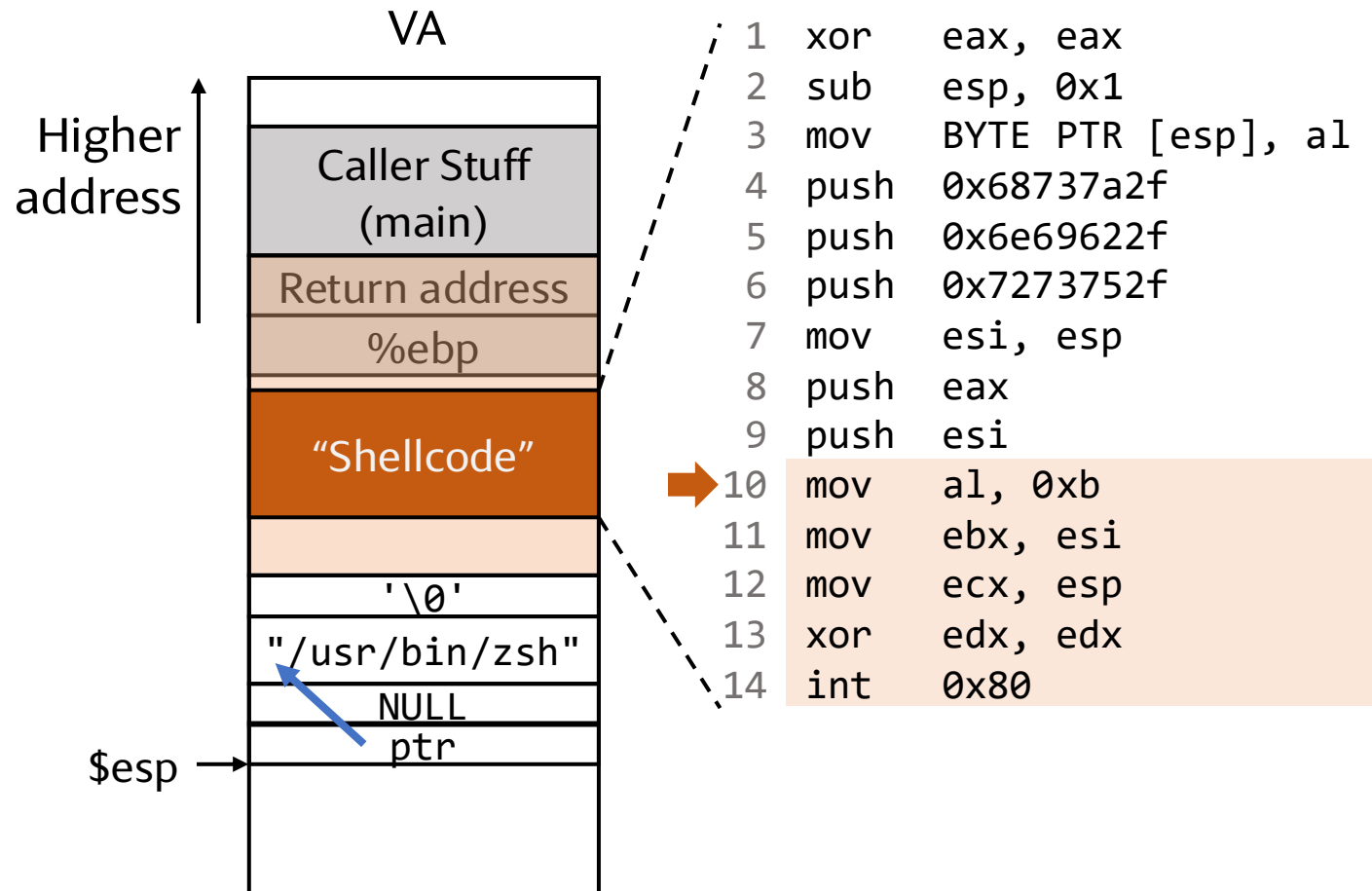
# Understanding The Shell Code



# Understanding The Shell Code



# Understanding The Shell Code



\$eax: 0xb

\$esi: <ptr to "/usr/bin/zsh">

\$ebx: <ptr to "/usr/bin/zsh">

\$ecx: <ptr to a pointer array>

\$edx: 0x0

Calling syscall 0xb (32-bit Linux):  
execve(  
 const char \*pathname, // ebx  
 char \* const argv[], // ecx  
 char \* const envp[] // edx  
);



execve(  
 "/usr/bin/zsh",  
 {"/usr/bin/zsh", NULL},  
 NULL  
);

# The setuid Bit

setuid: Allows users to run a program with the privileges of the program's owner

```
[zz9427@cara-d2:~]
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
[zz9427@cara-d2:~]
$ ls -l /etc/shadow
-rw-r----- 1 root shadow 2581 Sep  1 15:46 /etc/shadow
[zz9427@cara-d2:~]
$ ls -l (which cat)
-rwxr-xr-x 1 root root 39384 Aug 25 10:09 /usr/bin/cat*
[zz9427@cara-d2:~]
$ ls -l (which passwd)
-rwsr-xr-x 1 root root 64152 May 30 2024 /usr/bin/passwd*
```

# The sudoedit Vulnerability (CVE-2021-3156)

setuid: Allows users to run a program with the privileges of the program's owner

## CVE-2021-3156 Detail

### Description

Buffer overflow!

Sudo before 1.9.5p2 contains an off-by-one error that can result in a heap-based buffer overflow, which allows privilege escalation to root via "sudoedit -s" and a command-line argument that ends with a single backslash character.

### Metrics

CVSS Version 4.0

CVSS Version 3.x

CVSS Version 2.0

*NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.*

#### CVSS 3.x Severity and Vector Strings:



NIST: NVD

Base Score: 7.8 HIGH

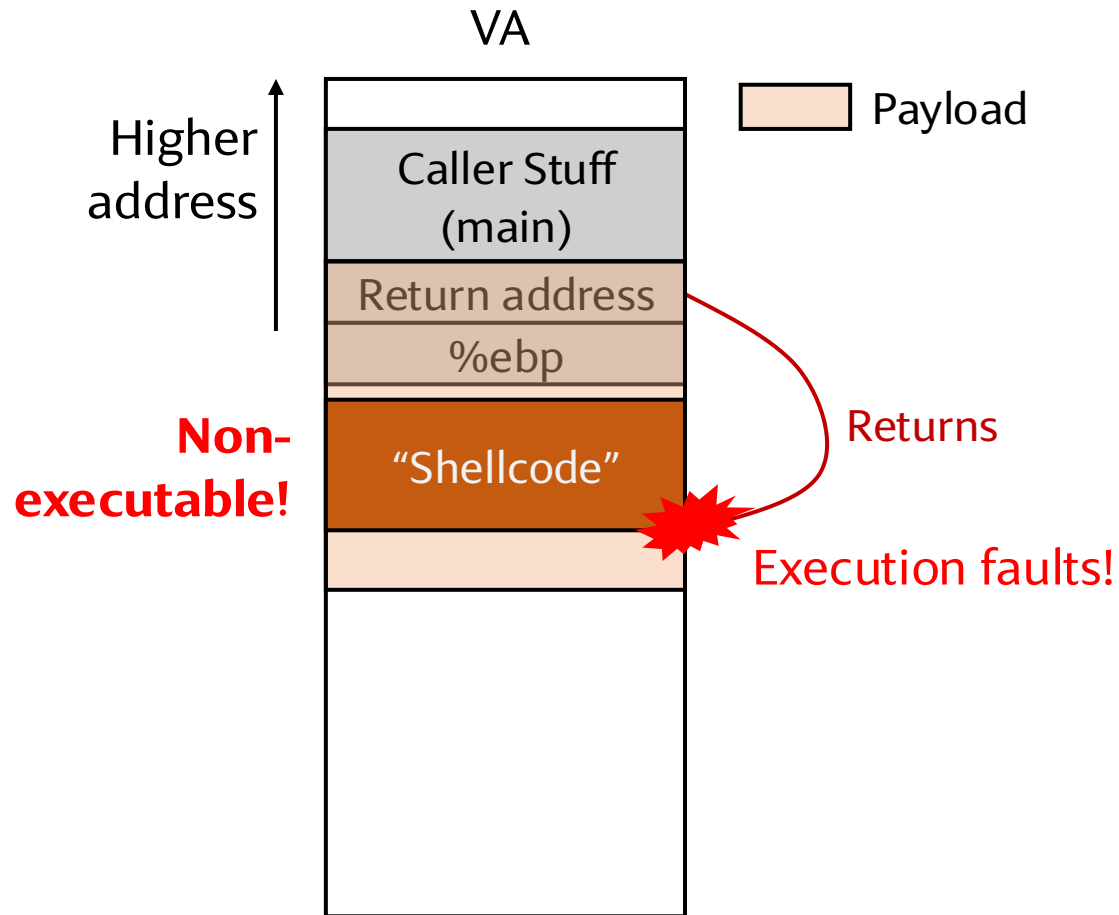
Vector: CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

ADP: CISA-ADP

Base Score: 7.8 HIGH

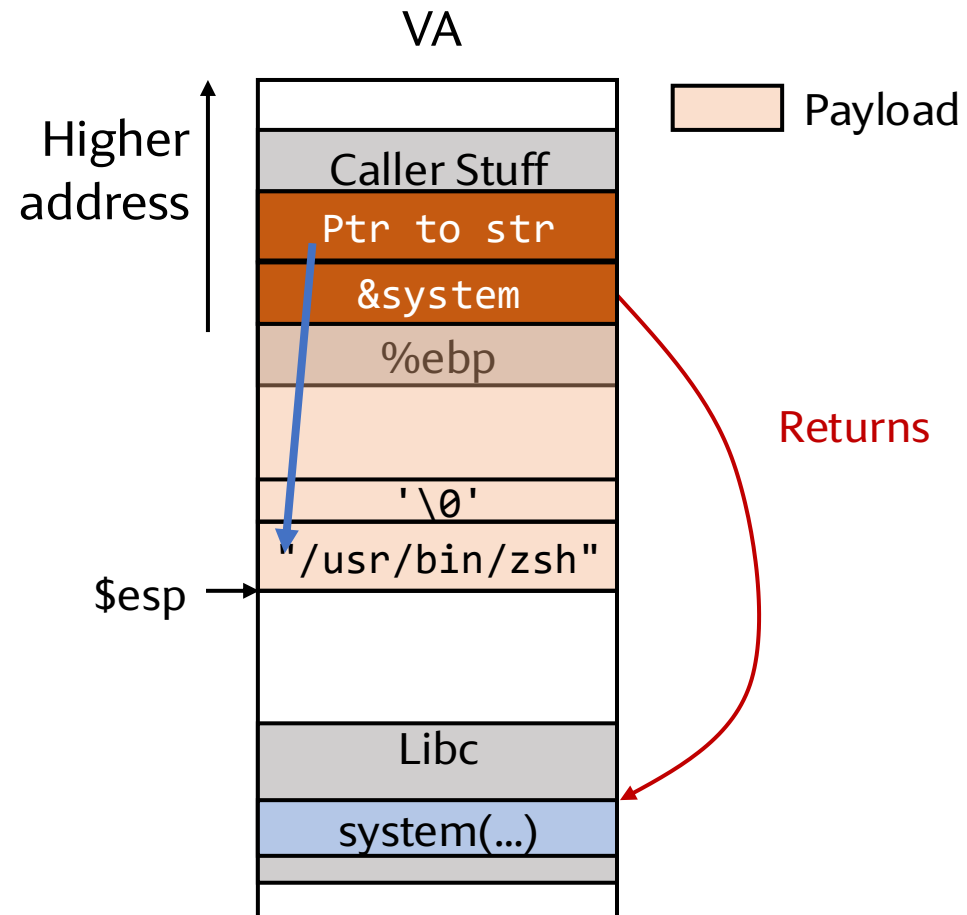
Vector: CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

# Mitigation: Data Execution Prevention (DEP)



**Common Policy:** a page is either writable or executable, but not both! (i.e.,  $W \oplus X$ )

# Code-Reuse Attack: Return-to-Library



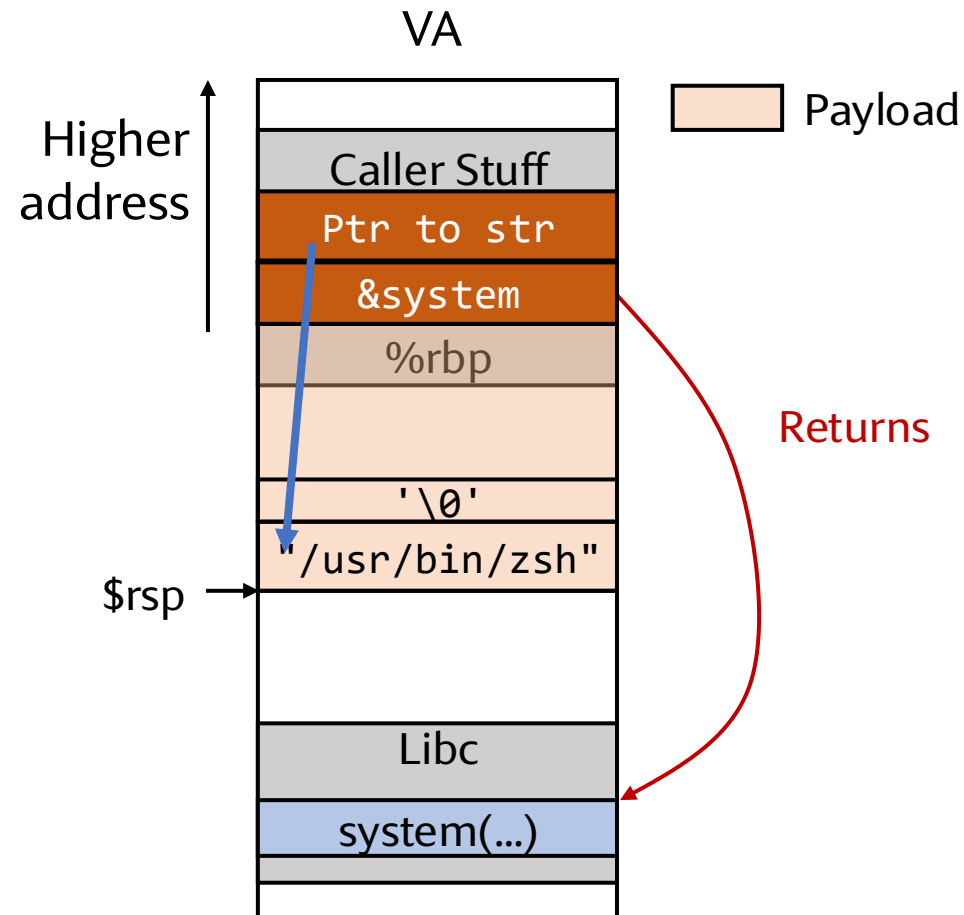
Libc's "system" function: executes a shell command  
`int system(const char *command);`

## Goal:

`system("/usr/bin/zsh");`

Works on x86 as arguments are stored on the stack

# Code-Reuse Attack: Return-to-Library



Libc's "system" function: executes a shell command  
`int system(const char *command);`

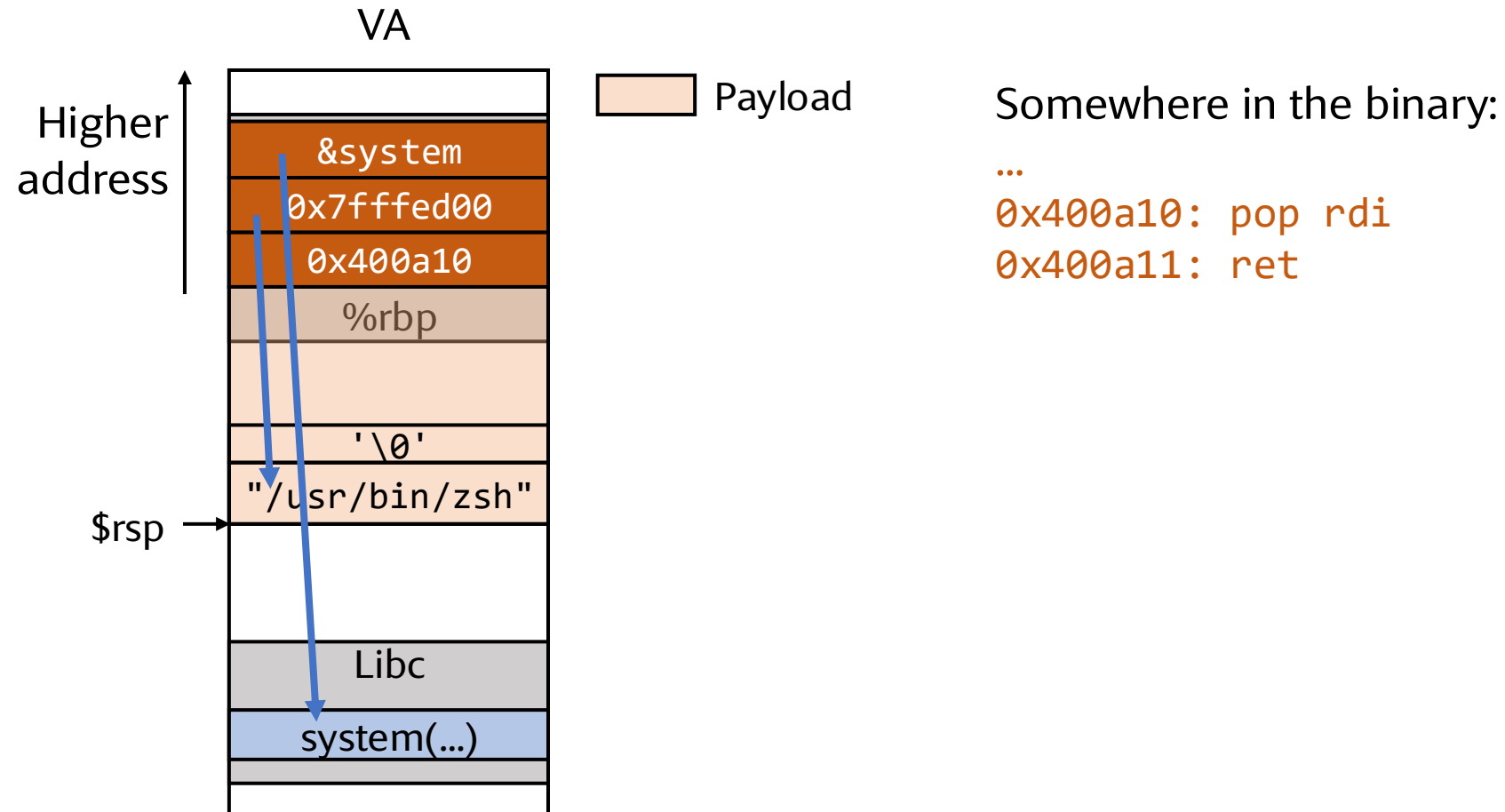
## Goal:

`system("/usr/bin/zsh");`

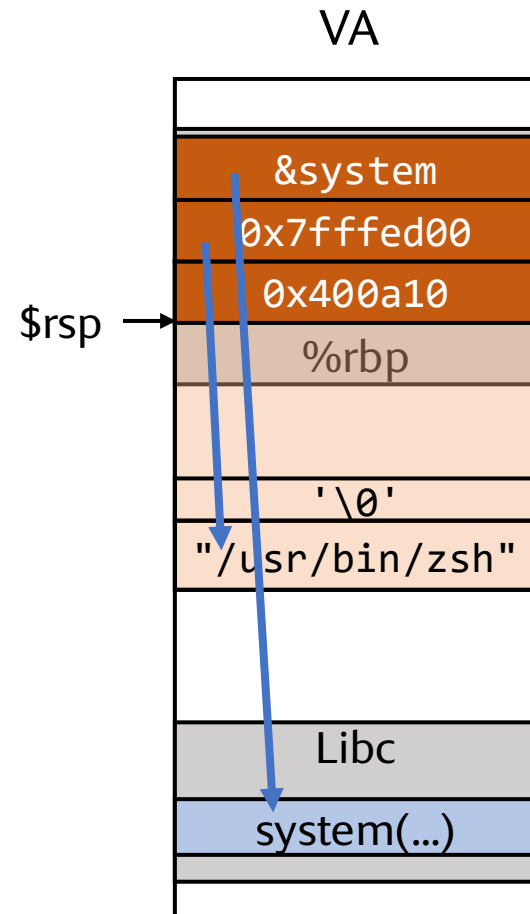
**Problem on x64:** On x64 Linux, the first argument is passed through register `$rdi`, which must hold a ptr to `"/usr/bin/zsh"`. How to set `$rdi`?



# Return-Oriented Programming (ROP)



# Return-Oriented Programming (ROP)



 Payload

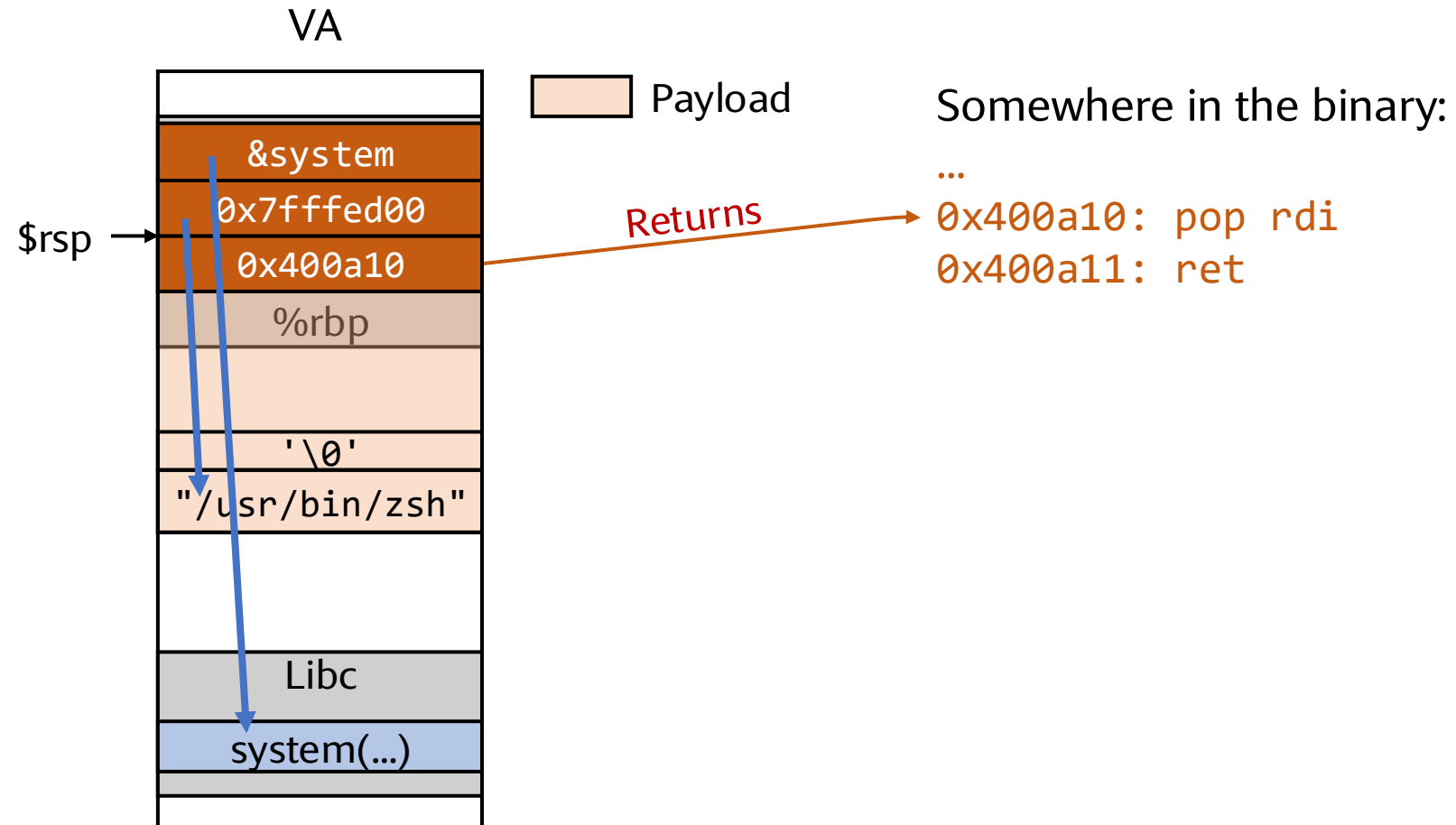
Somewhere in the binary:

...

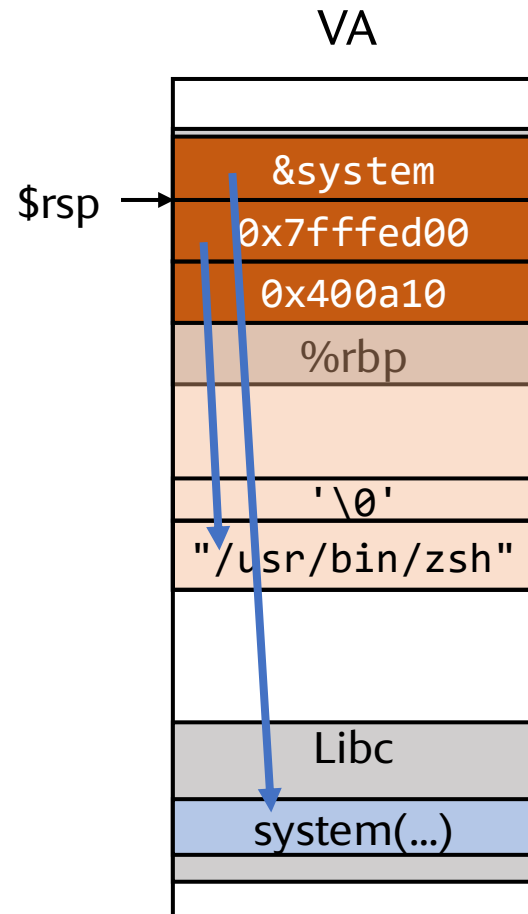
0x400a10: pop rdi

0x400a11: ret

# Return-Oriented Programming (ROP)



# Return-Oriented Programming (ROP)



 Payload

Somewhere in the binary:

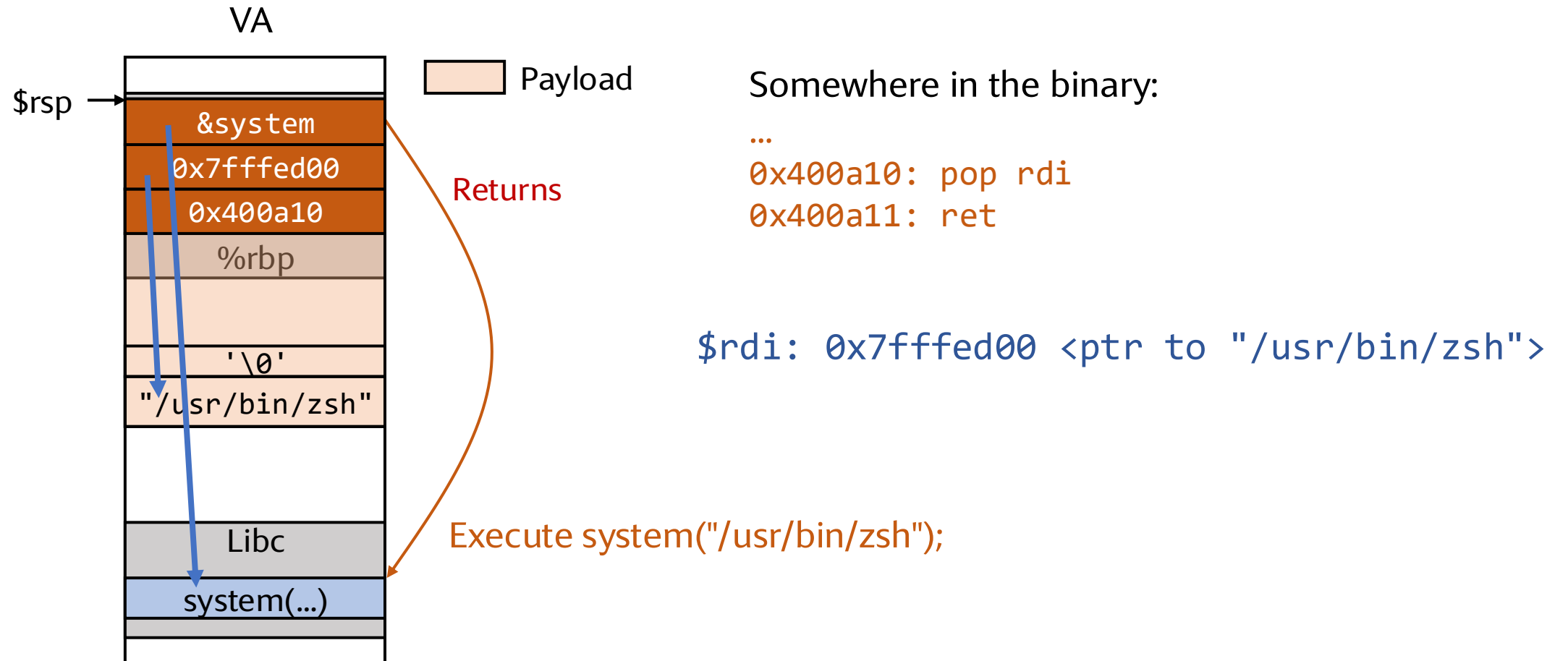
...

0x400a10: pop rdi

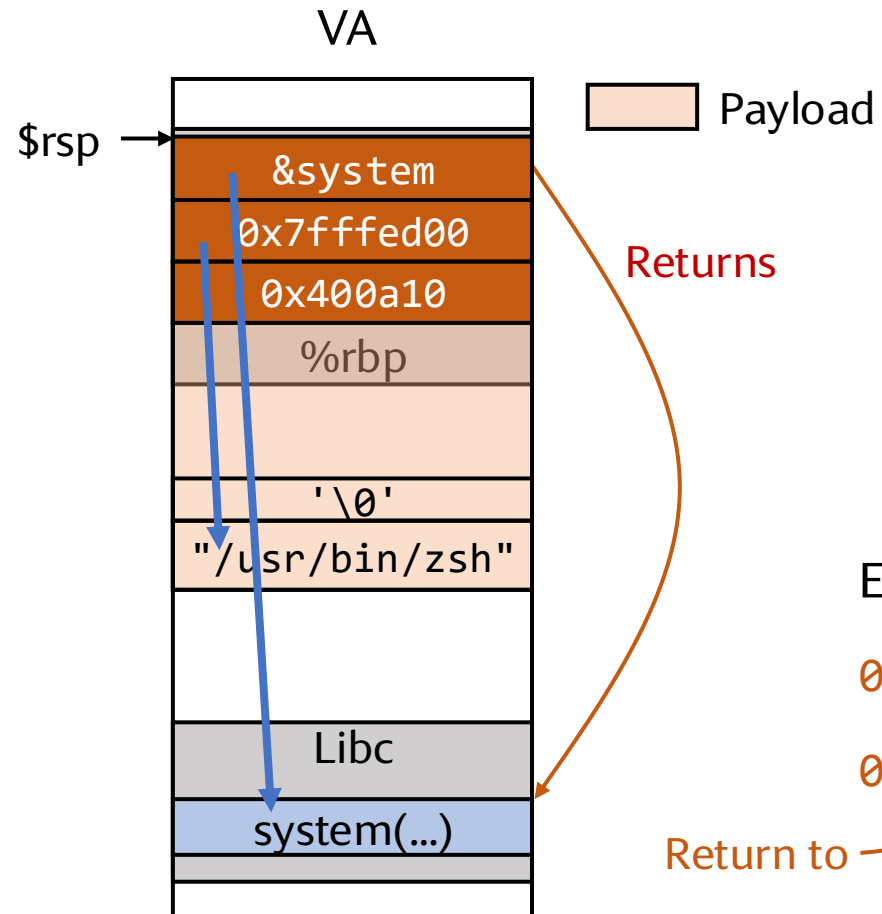
0x400a11: ret

\$rdi: 0x7fffed00 <ptr to "/usr/bin/zsh">

# Return-Oriented Programming (ROP)



# Return-Oriented Programming (ROP)



Somewhere in the binary:

```
...  
0x400a10: pop rdi  
0x400a11: ret
```



"\x5f\xc3"

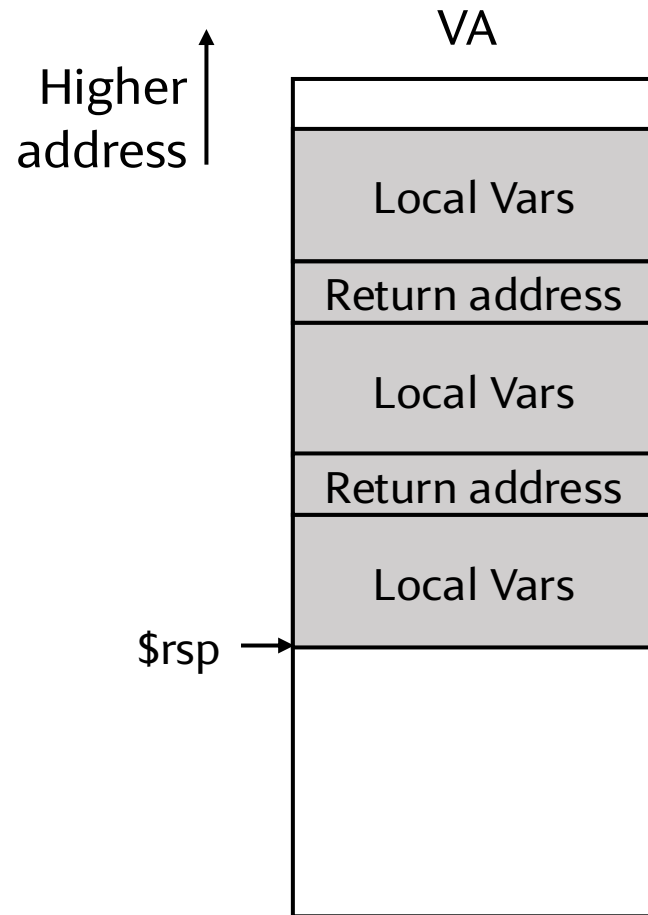
Exploiting variable-length, unaligned x86 instructions:

```
0f 5f c3: maxps xmm0, xmm3 // compiler emitted
```

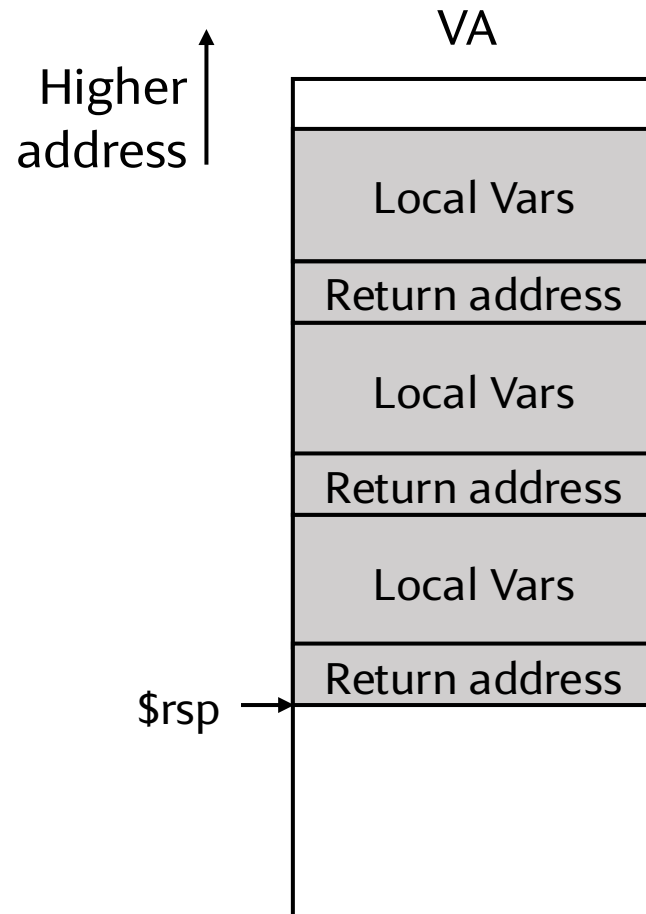
```
0f 5f c3: pop rdi; ret // misused
```

Return to →

# ROP Weird Machine



# ROP Weird Machine

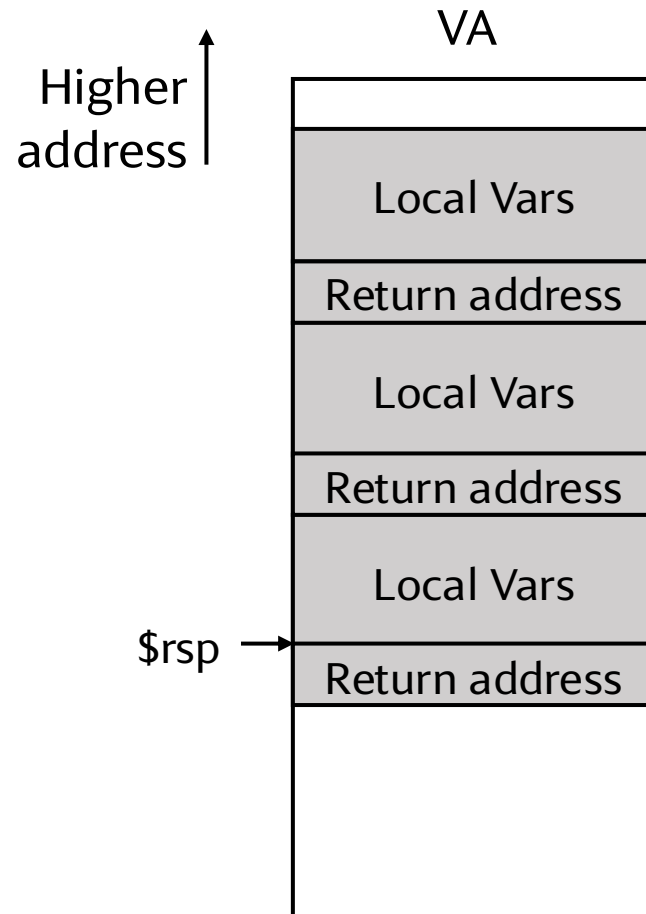


`call <addr>:`

Push the return address to the stack, jump to <addr>



# ROP Weird Machine



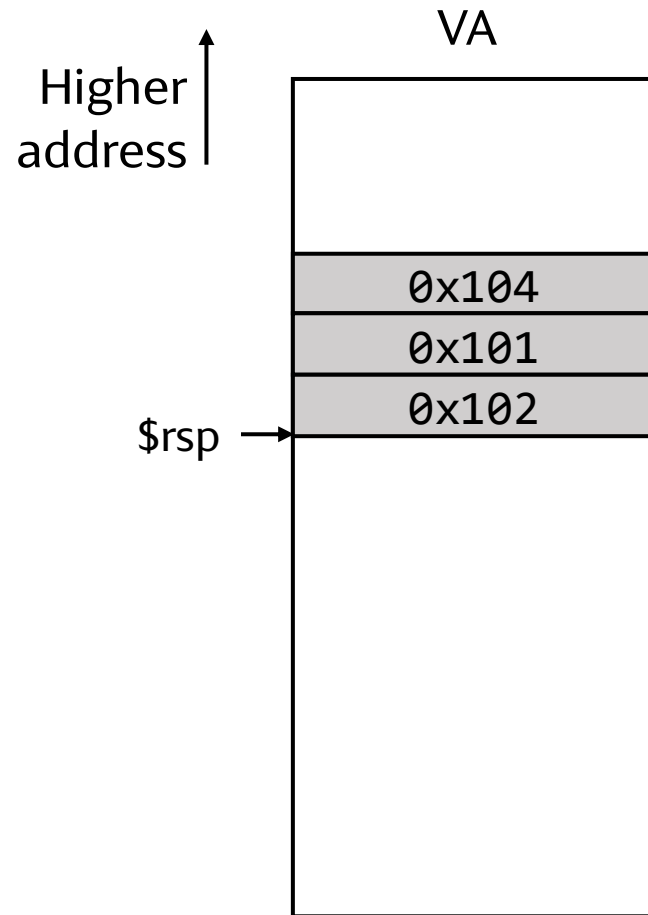
`call <addr>:`

Push the return address to the stack, jump to <addr>

`ret:`

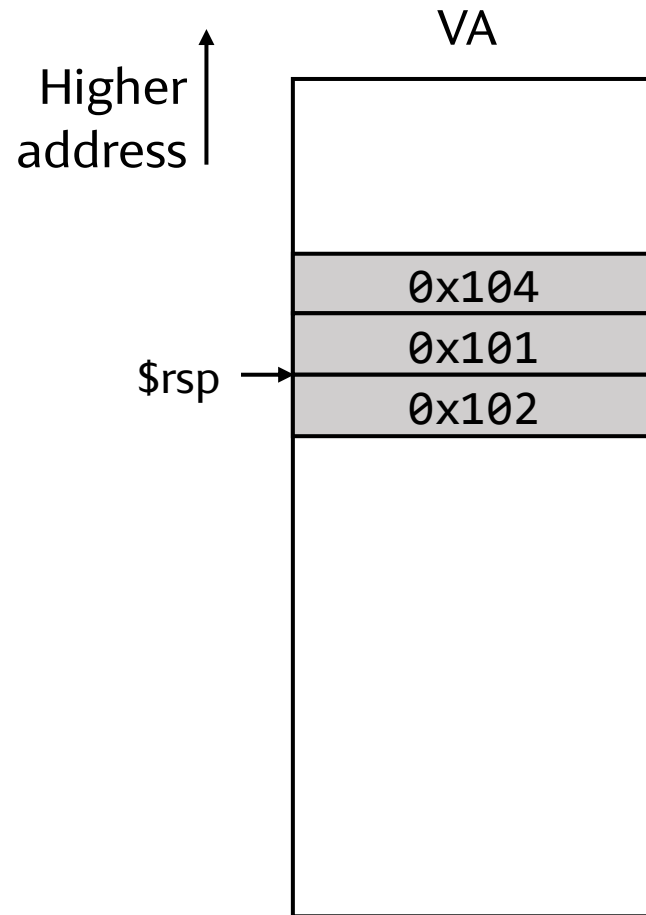
Pop a value from the stack, **treat it as an address**, jump to there

# ROP Weird Machine



```
0x100: ret
0x101: ret
0x102: ret
0x103: ret
0x104: ret
0x105: ret
```

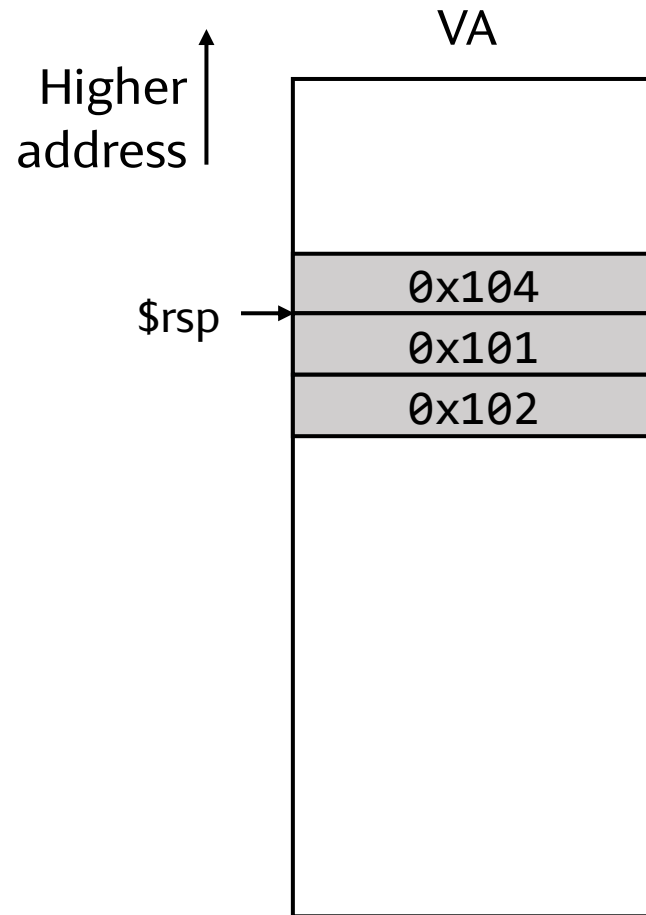
# ROP Weird Machine



\$rip →

|        |     |
|--------|-----|
| 0x100: | ret |
| 0x101: | ret |
| 0x102: | ret |
| 0x103: | ret |
| 0x104: | ret |
| 0x105: | ret |

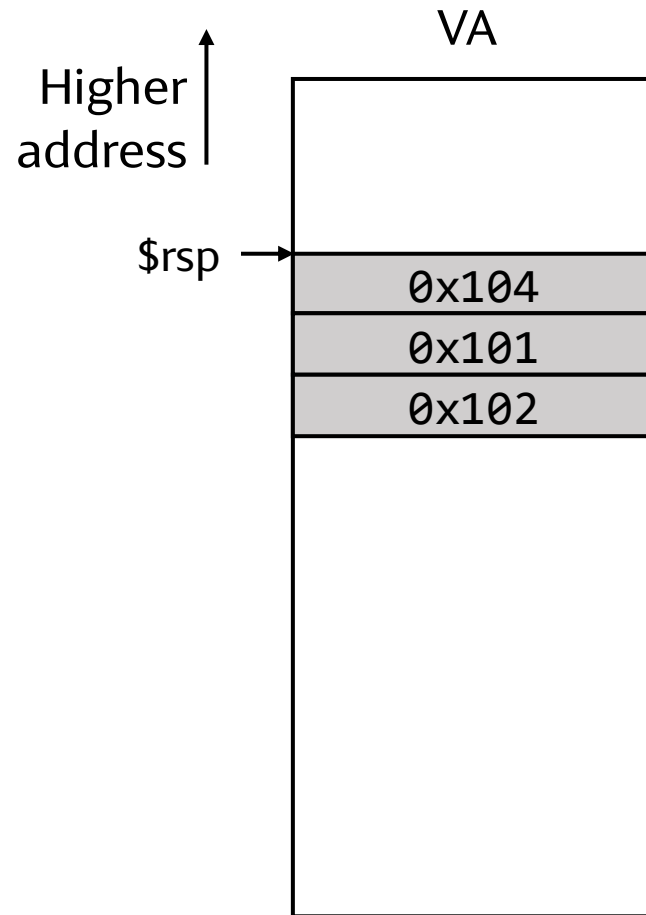
# ROP Weird Machine



\$rip →

|            |
|------------|
| 0x100: ret |
| 0x101: ret |
| 0x102: ret |
| 0x103: ret |
| 0x104: ret |
| 0x105: ret |

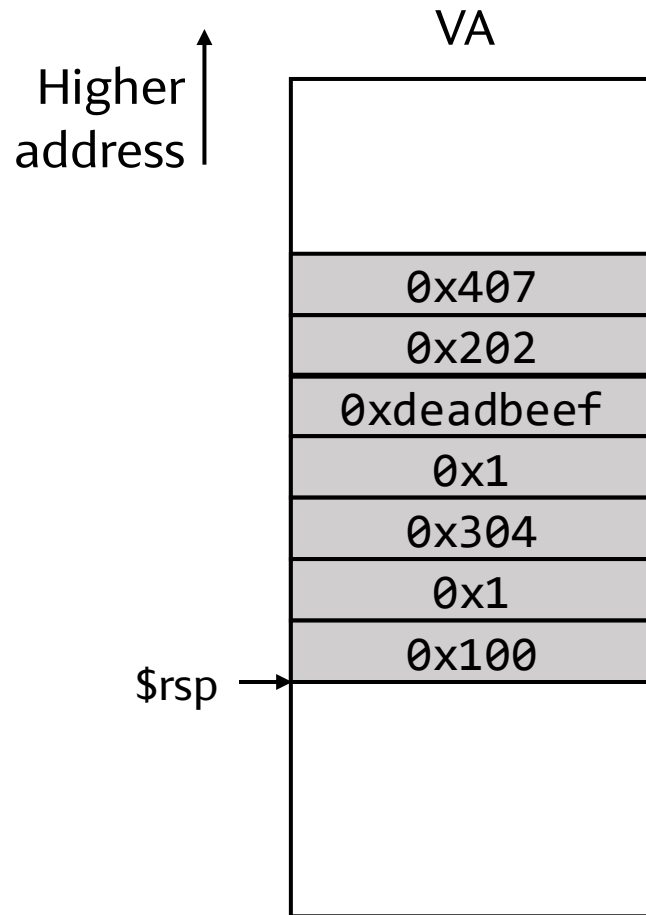
# ROP Weird Machine



```
0x100: ret
0x101: ret
0x102: ret
0x103: ret
$rip → 0x104: ret
0x105: ret
```

We can program the control flow using the stack content!

# ROP Weird Machine – Add Two Numbers



```
0x100: pop rdi
0x101: ret
```

Set \$rdi

...

```
0x202: add rcx, rdi
0x205: ret
```

\$rcx=\$rcx+\$rdi

...

```
0x304: pop rcx
0x305: pop rbx
0x306: ret
```

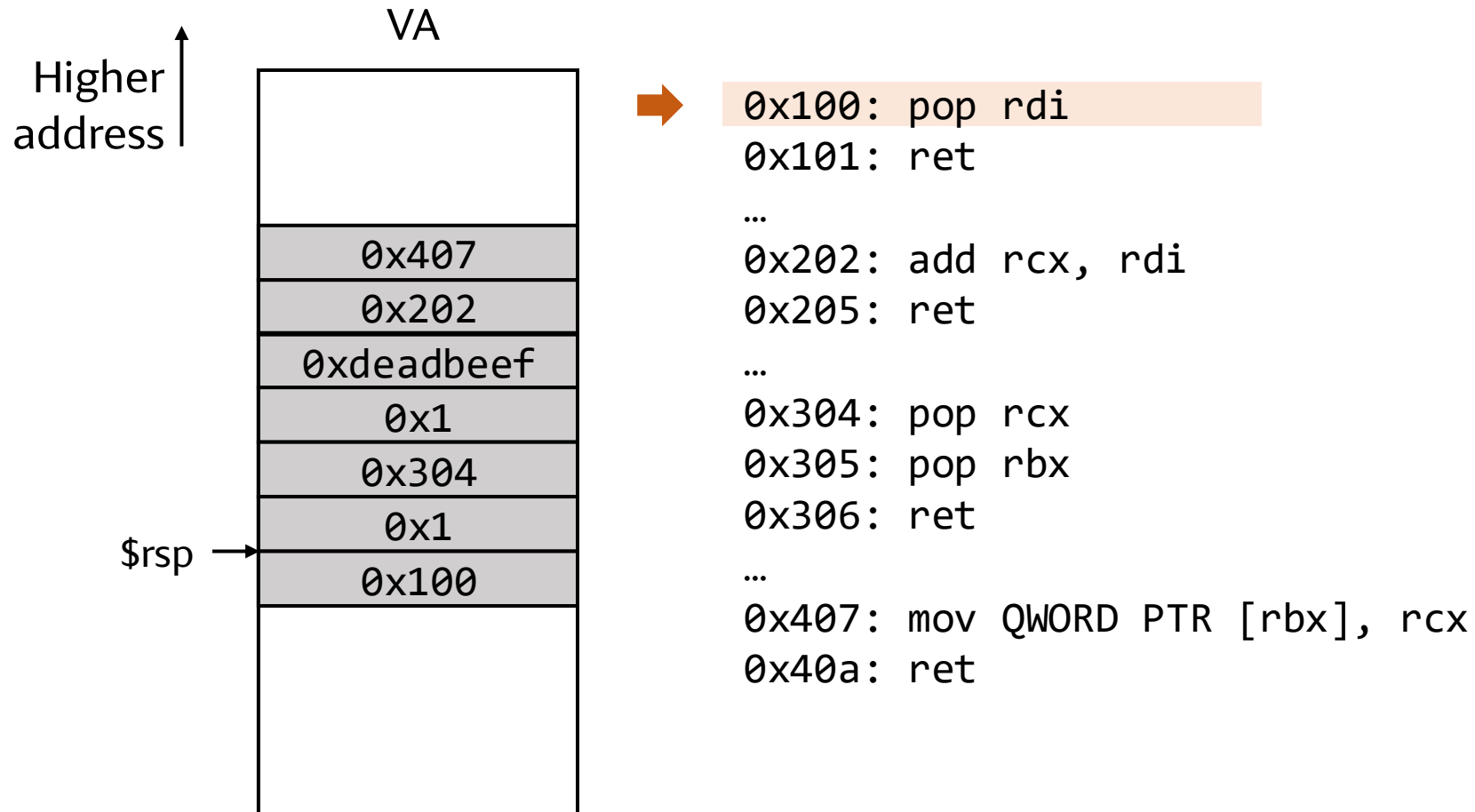
Set \$rcx, \$rbx

...

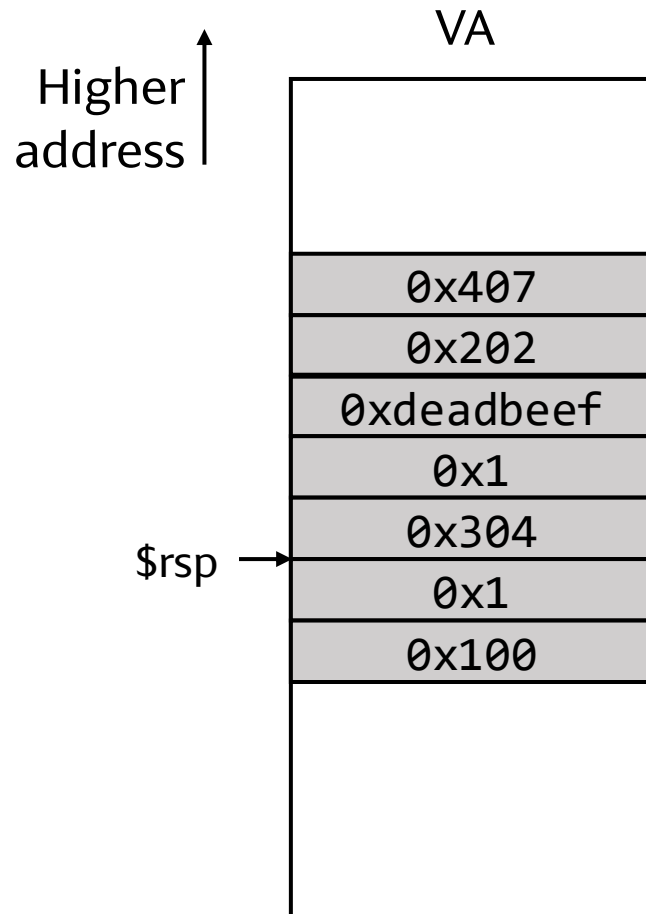
```
0x407: mov QWORD PTR [rbx], rcx
0x40a: ret
```

Store \$rcx->[\$rbx]

# ROP Weird Machine – Add Two Numbers



# ROP Weird Machine – Add Two Numbers



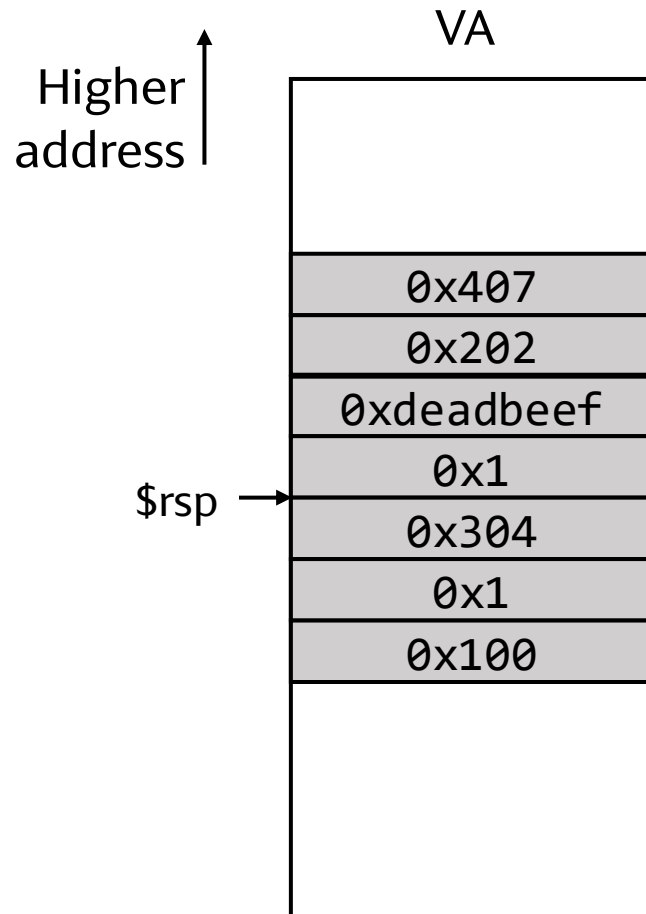
```
0x100: pop rdi  
0x101: ret
```

**\$rdi: 0x1**

```
...  
0x202: add rcx, rdi  
0x205: ret  
...  
0x304: pop rcx  
0x305: pop rbx  
0x306: ret  
...  
0x407: mov QWORD PTR [rbx], rcx  
0x40a: ret
```



# ROP Weird Machine – Add Two Numbers



0x100: pop rdi  
0x101: ret

\$rdi: 0x1

...

0x202: add rcx, rdi  
0x205: ret

...

0x304: pop rcx

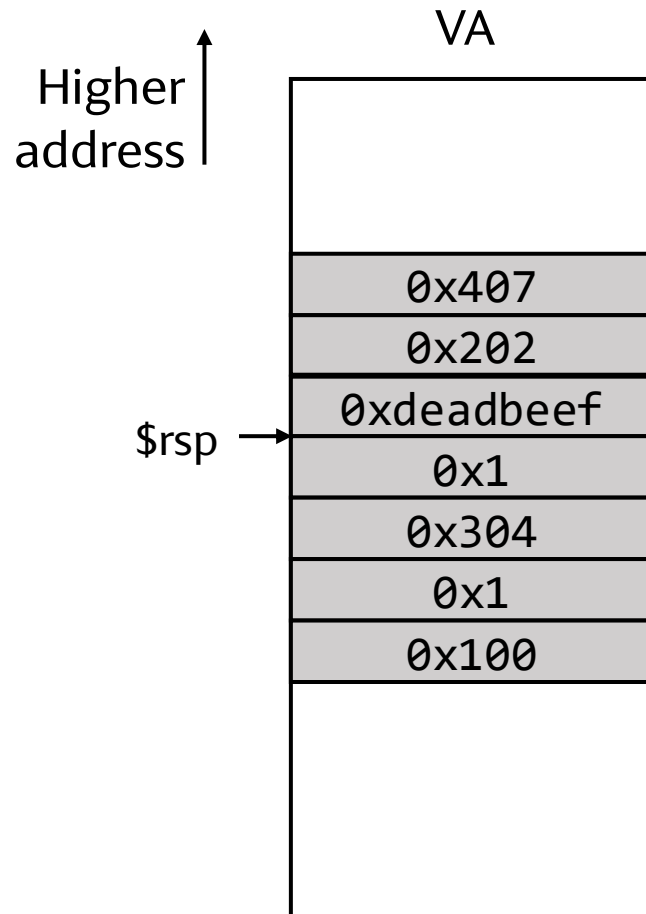
0x305: pop rbx

0x306: ret

...

0x407: mov QWORD PTR [rbx], rcx  
0x40a: ret

# ROP Weird Machine – Add Two Numbers



0x100: pop rdi  
0x101: ret

\$rdi: 0x1

\$rcx: 0x1

...

0x202: add rcx, rdi  
0x205: ret

...

0x304: pop rcx



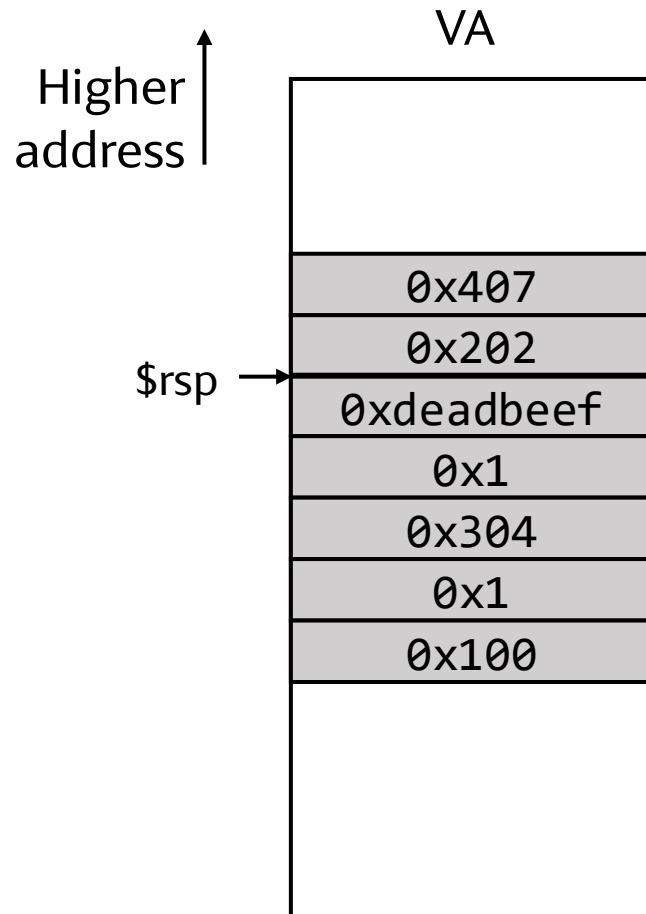
0x305: pop rbx

0x306: ret

...

0x407: mov QWORD PTR [rbx], rcx  
0x40a: ret

# ROP Weird Machine – Add Two Numbers



0x100: pop rdi  
0x101: ret

...

0x202: add rcx, rdi  
0x205: ret

...

0x304: pop rcx  
0x305: pop rbx



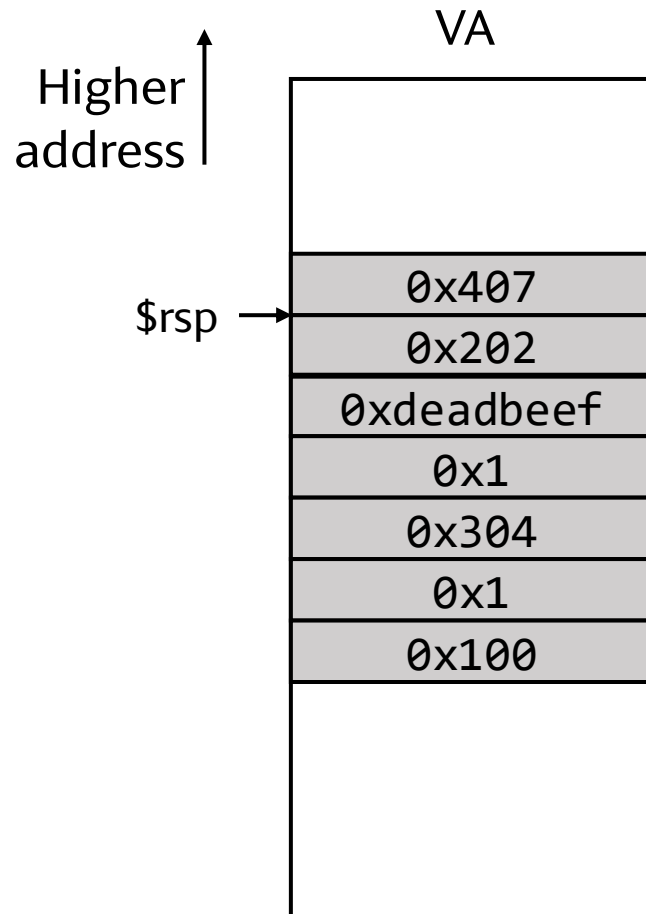
0x306: ret

...

0x407: mov QWORD PTR [rbx], rcx  
0x40a: ret

**\$rdi: 0x1**  
**\$rcx: 0x1**  
**\$rbx: 0xdeadbeef**

# ROP Weird Machine – Add Two Numbers



```
0x100: pop rdi
0x101: ret
```

...

```
0x202: add rcx, rdi
0x205: ret
```

...

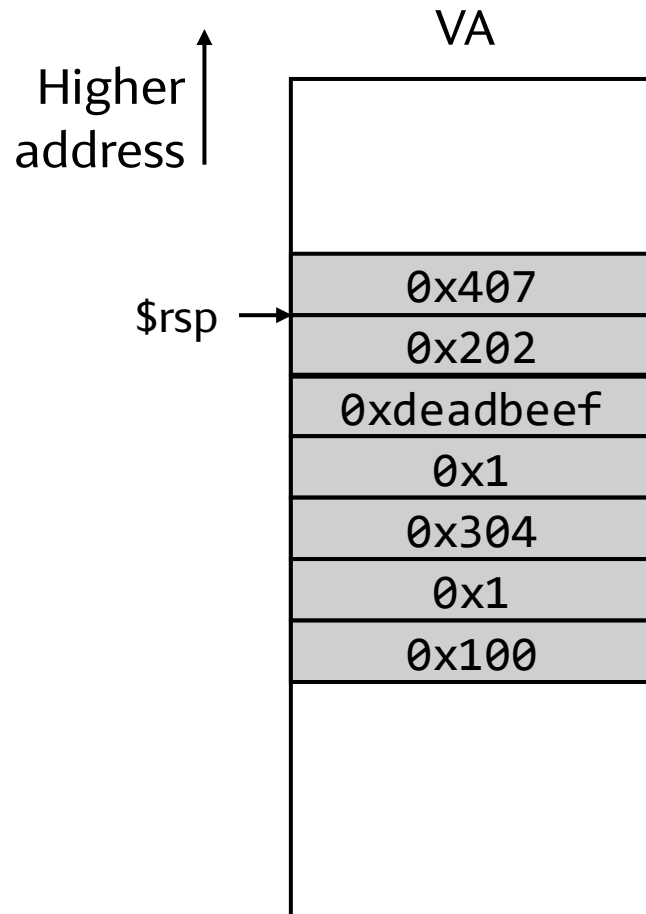
```
0x304: pop rcx
0x305: pop rbx
0x306: ret
```

...

```
0x407: mov QWORD PTR [rbx], rcx
0x40a: ret
```

```
$rdi: 0x1
$rcx: 0x1
$rbx: 0xdeadbeef
```

# ROP Weird Machine – Add Two Numbers



```
0x100: pop rdi
0x101: ret
```

...

```
0x202: add rcx, rdi
```

```
0x205: ret
```

...

```
0x304: pop rcx
```

```
0x305: pop rbx
```

```
0x306: ret
```

...

```
0x407: mov QWORD PTR [rbx], rcx
```

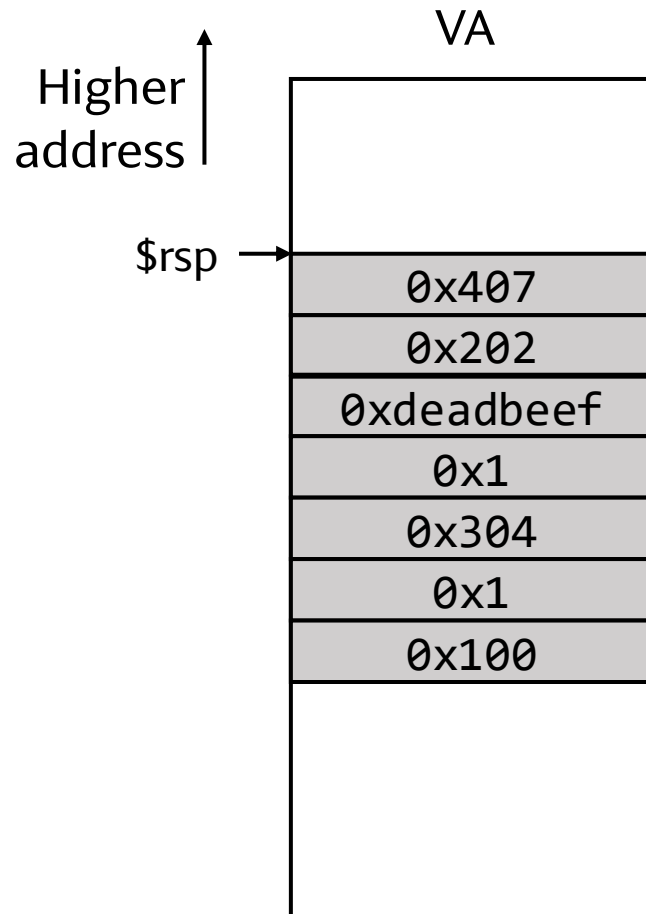
```
0x40a: ret
```

`$rdi: 0x1`

`$rcx: 0x2`

`$rbx: 0xdeadbeef`

# ROP Weird Machine – Add Two Numbers



```
0x100: pop rdi
0x101: ret
```

...

```
0x202: add rcx, rdi
0x205: ret
```

...

```
0x304: pop rcx
0x305: pop rbx
0x306: ret
```

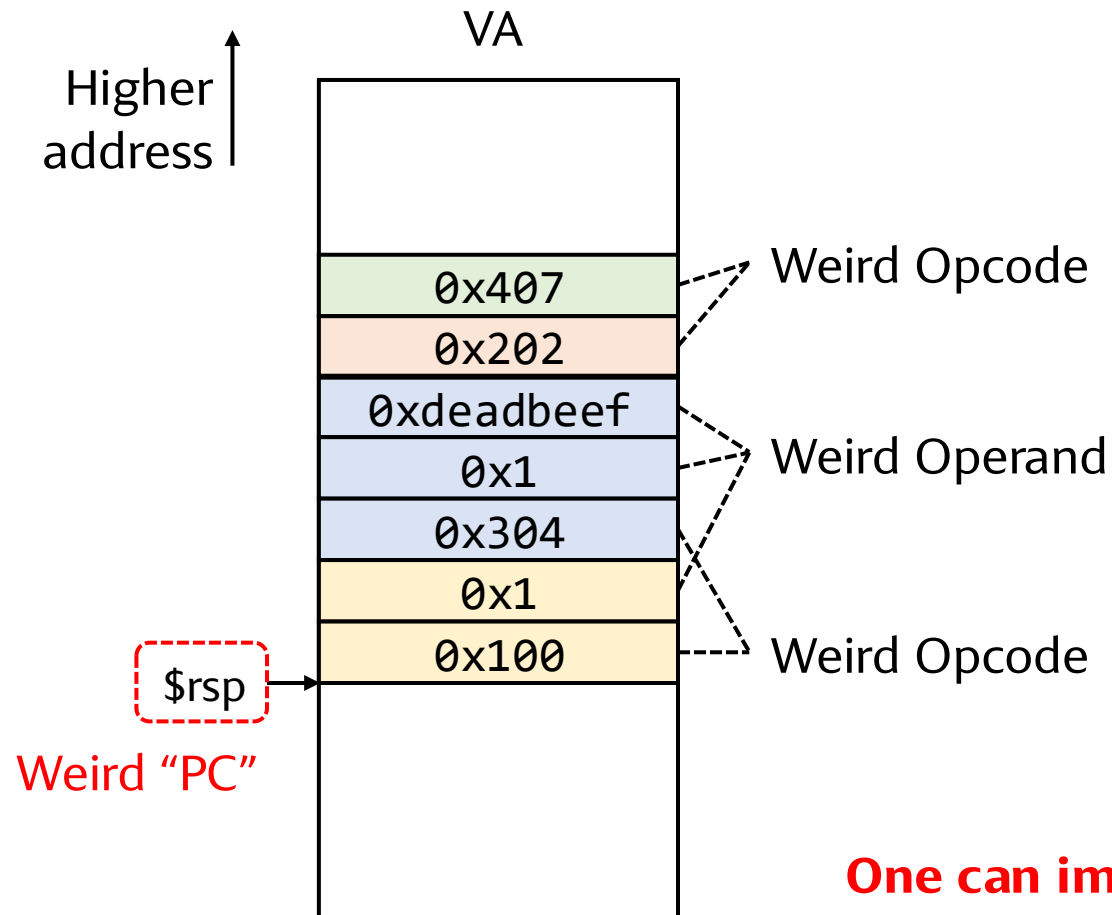
...

➔ 0x407: mov QWORD PTR [rbx], rcx Store the result to memory

```
0x40a: ret
```

```
$rdi: 0x1
$rcx: 0x2
$rbx: 0xdeadbeef
```

# The Weird ISA Based on ROP



## Weird Instructions

```
0x100: pop rdi
0x101: ret
```

...

```
0x202: add rcx, rdi
0x205: ret
```

...

```
0x304: pop rcx
0x305: pop rbx
0x306: ret
```

...

```
0x407: mov QWORD PTR [rbx], rcx
0x40a: ret
```

**One can implement branches!**

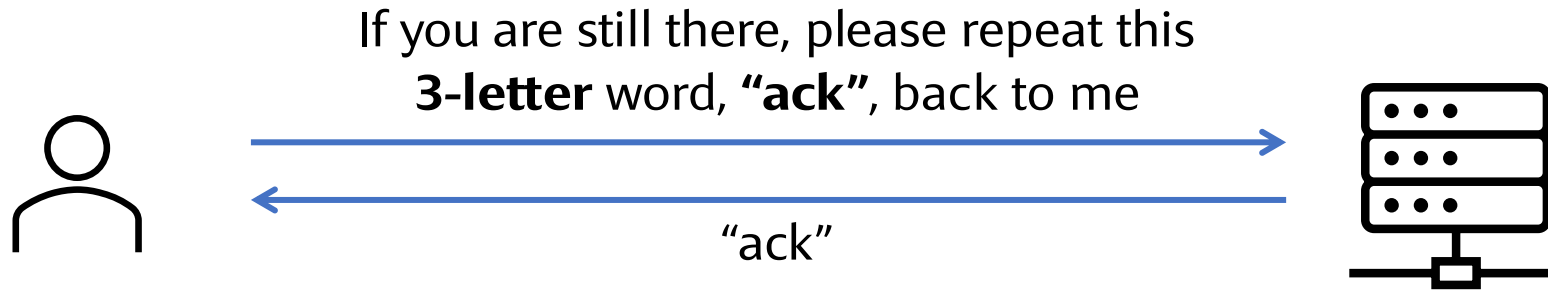
# Spatial Memory Safety

## **De-referencing an out-of-bound pointer:**

- Out-of-bound writes → Buffer overflow
  - Compromise program integrity
- Out-of-bound reads → Buffer over-read
  - Information leakage: A high-profile example: Heartbleed



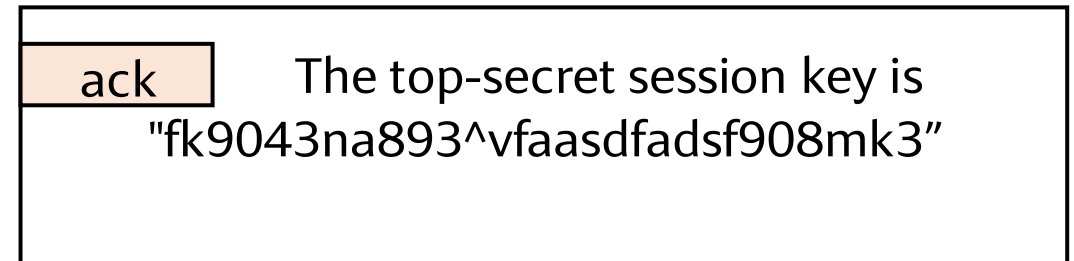
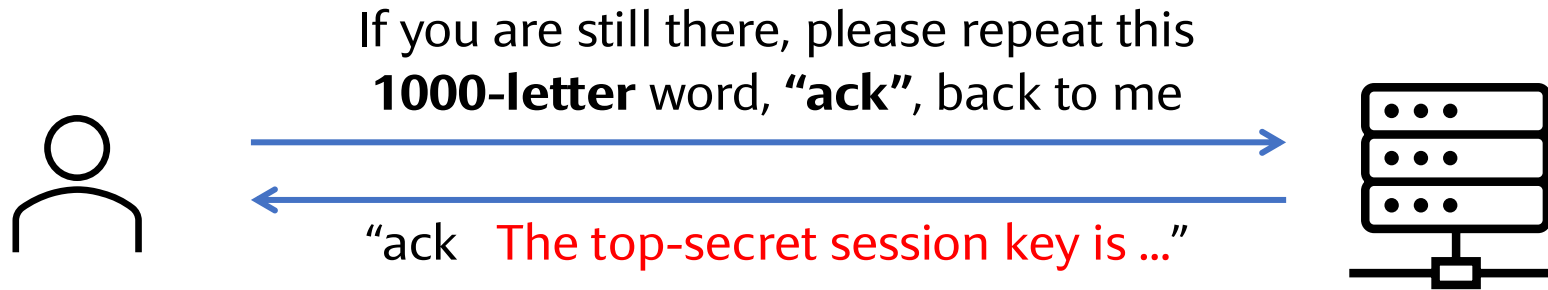
# Heartbleed (CVE-2014-0160)



|     |                                                               |
|-----|---------------------------------------------------------------|
|     |                                                               |
| ack | The top-secret session key is<br>"fk9043na893^vfaasdfs908mk3" |
|     |                                                               |

Memory

# Heartbleed (CVE-2014-0160)



Memory

# Temporal Memory Safety - Use-after-Free

De-referencing a dangling pointer that points to a deleted object

```
1 typedef struct {
2     int id;
3     char secret[60];
4 } Obj; // 64B
5
6 char *buf = malloc(64);
7 // ...
8 free(buf);
9
10 Obj *obj = malloc(sizeof(Obj));
11 memcpy(obj->secret, "secret", 7);
12
13 printf("%s\n", &buf[4]);
```

```
1 typedef void (*myfunc)(void);
2 typedef struct {
3     int id;
4     myfunc func;
5 } Obj; // 16B
6
7 char *buf = malloc(16);
8 // ...
9 free(buf);
10
11 Obj *obj = malloc(sizeof(Obj));
12
13 buf[...] = ...;
14 obj->func(); // compromised!
```

# Temporal Memory Safety – Double-Free

The only thing worse than double-free is double-dipping!

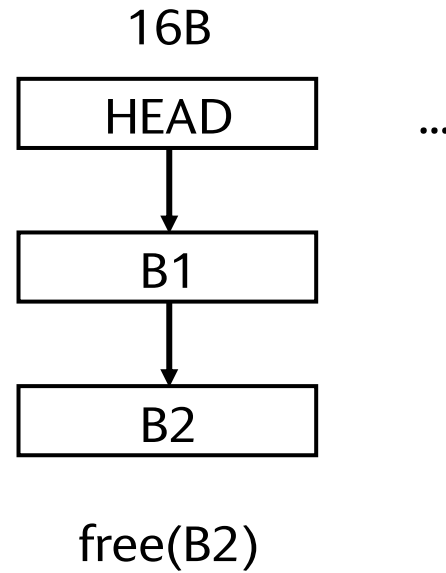
```
1 char *a = malloc(10); // 0x5683fe74e6b0
2 char *b = malloc(10); // 0x5683fe74e6d0
3 char *c = malloc(10); // 0x5683fe74e6f0
4
5 free(a);
6 free(b);
7 free(a);
8
9 char *d = malloc(10); // 0x5683fe74e6b0
10 char *e = malloc(10); // 0x5683fe74e6d0
11 char *f = malloc(10); // 0x5683fe74e6b0
```

But why? Is malloc really that lazy?

\*Historical glibc fastbin double-free (glibc < 2.43; tcache disabled)

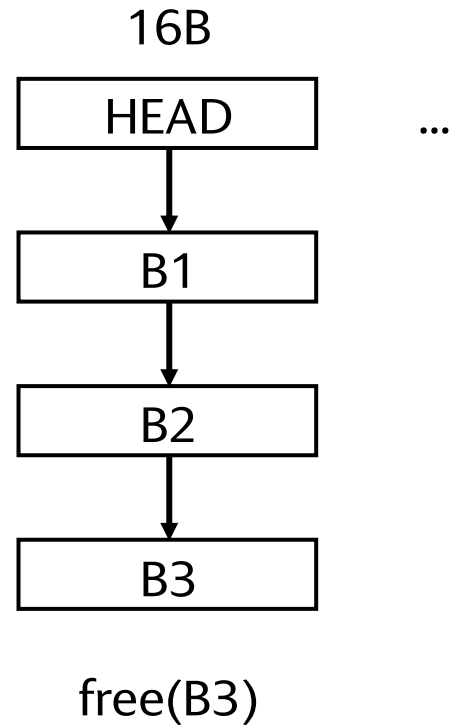
# Fastbins

A small software cache for holding recently freed small memory chunks



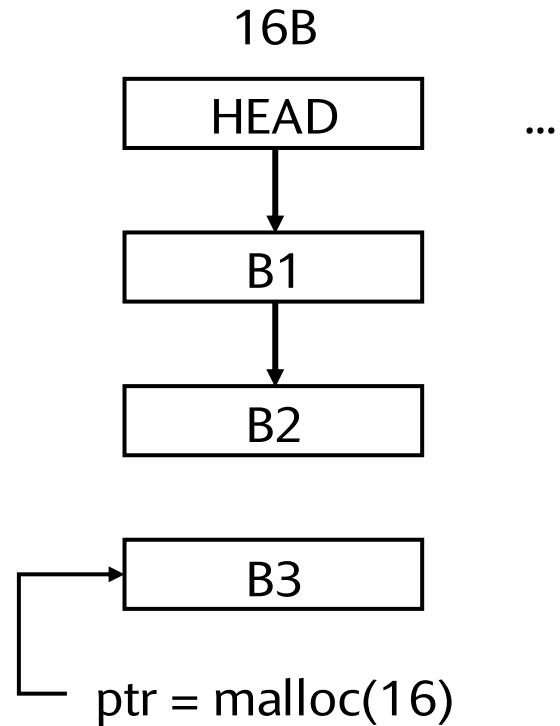
# Fastbins

A small software cache for holding recently freed small memory chunks



# Fastbins

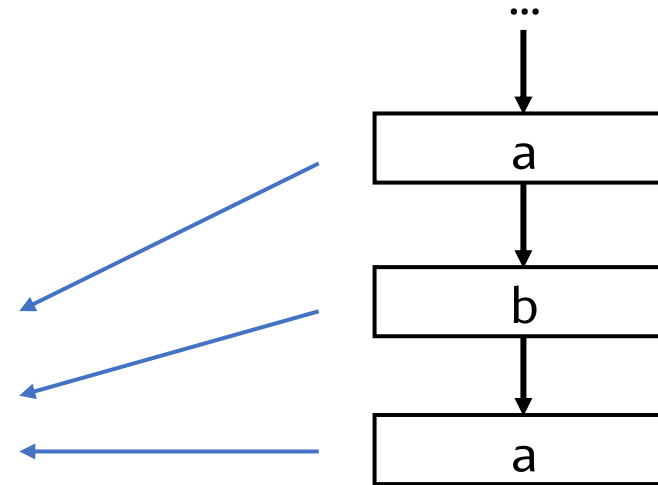
A small software cache for holding recently freed small memory chunks



# Temporal Memory Safety – Double-Free

The only thing worse than double-free is double-dipping!

```
1 char *a = malloc(10); // 0x5683fe74e6b0
2 char *b = malloc(10); // 0x5683fe74e6d0
3 char *c = malloc(10); // 0x5683fe74e6f0
4
5 free(a);
6 free(b);
7 free(a);
8
9 char *d = malloc(10); // 0x5683fe74e6b0
10 char *e = malloc(10); // 0x5683fe74e6d0
11 char *f = malloc(10); // 0x5683fe74e6b0
```



\*Historical glibc fastbin double-free (glibc < 2.43; tcache disabled)